Шинжлэх Ухаан, Технологийн Их Сургууль
Мэдээлэл, Холбооны Технологийн Сургууль

М.Түвшинжаргал

# Үүлэн орчинд суурилсан спорт заалны цаг захиалгын 3 давхаргат системийн загварчлал, хэрэгжилт

Бакалаврын төгсөлтийн ажил

Улаанбаатар хот

Шинжлэх Ухаан, Технологийн Их Сургууль
Мэдээлэл, Холбооны Технологийн Сургууль

Мэдээллийн технологийн тэнхим

# Үүлэн орчинд суурилсан спорт заалны цаг захиалгын 3 давхаргат системийн загварчлал, хэрэгжилт

Мэргэжлийн индекс: D061304
Мэргэжил: Мэдээллийн технологи
*Удирдагч:* доктор Д.Эрдэнэтуяа
*Зөвлөгч:*
*Гүйцэтгэгч:* М.Түвшинжаргал

Улаанбаатар хот
2025 он 6 сар

Батлав. Мэдээллийн технологийн тэнхимын эрхлэгч:

..... //

Удирдагч: ..... /доктор Д.Эрдэнэтуяа/

## ТӨГСӨЛТИЙН АЖЛЫН ТӨЛӨВЛӨГӨӨ

**Төгсөлтийн ажлын сэдэв:**
**Монгол**: Үүлэн орчинд суурилсан спорт заалны цаг захиалгын 3 давхаргат системийн загварчлал, хэрэгжилт
**Англи**: Design and Implementation of a Three-Tier Sports Hall Booking System Based on Cloud Environment

**Төслийн зорилго**: Өмнөговь аймгийн Даланзадгад хотын спорт заалнуудын цаг захиалгыг цахимжуулах, хэрэглэгчдэд хялбар, хурдан захиалга хийх боломж олгох Flutter мобайл аппликейшн хөгжүүлэх.

**Оюутны овог нэр, код**: М.Түвшинжаргал/B221940619/

| № | Ажлын бүлэг, хэсгийн нэр | эзлэх хувь | дуусах хугацаа |
|---|---|---|---|
| Бүлэг №1. Оршил ба судалгааны үндэслэл | | | |
| 1 | 1.1 Судалгааны үндэслэл, асуудлын тодорхойлолт<br>1.2 Зорилго, зорилтууд<br>1.3 Ажлын ач холбогдол, хамрах хүрээ | 20% | III.13 |
| Бүлэг №2. Онолын үндэс ба технологийн судалгаа | | | |
| 2 | 2.1 Мобайл апп хөгжүүлэлтийн орчин үеийн арга хандлага<br>2.2 Flutter фреймворк ба Dart хэл<br>2.3 Firebase бэкэнд үйлчилгээ<br>2.4 Цаг захиалгын системийн онолын үндэс | 30% | IV.10 |
| Бүлэг №3. Системийн зохион байгуулалт ба хэрэгжүүлэлт | | | |
| 3 | 3.1 Системийн шаардлагын дүн шинжилгээ<br>3.2 Системийн архитектур<br>3.3 Өгөгдлийн сангийн зохион байгуулалт<br>3.4 Flutter апп-ын хэрэгжүүлэлт<br>3.5 Туршилт ба үр дүнгийн үнэлгээ | 50% | V.22 |
| Ерөнхий дүгнэлт | | | |

Төлөвлөгөөг боловсруулсан оюутан: ..... /М.Түвшинжаргал/

## ТӨГСӨЛТИЙН АЖЛЫН ЯВЦ

| № | Хийж гүйцэтгэсэн ажил | Биелсэн хугацаа | Удирдагчийн гарын үсэг |
|---|---|---|---|
| 1 | Бүлэг 1: Оршил, судалгааны үндэслэл боловсруулсан | 2025-03-13 | |
| 2 | Бүлэг 2: Онолын үндэс, технологийн судалгаа хийсэн | 2025-04-10 | |
| 3 | Бүлэг 3: Систем зохион байгуулж, Flutter апп хөгжүүлсэн | 2025-05-15 | |
| 4 | Туршилт хийж, дүгнэлт боловсруулсан | 2025-05-22 | |

Ажлын товч дүгнэлт

.....

.....

.....

.....

.....

.....

.....

Удирдагч: ..... /доктор Д.Эрдэнэтуяа/

ЗӨВШӨӨРӨЛ

Оюутан М.Түвшинжаргалгын бичсэн төгсөлтийн ажлыг УШК-д хамгаалуулахаар тодорхойлов.

Салбарын эрхлэгч: ..... //

# Зохиогчийн эрх хамгаалал

Миний бие М.Түвшинжаргал, Үүлэн орчинд суурилсан спорт заалны цаг захиалгын 3 давхаргат системийн загварчлал, хэрэгжилт сэдэвт энэ ажил нь минийх бөгөөд дараахыг нотолж байна. Үүнд:

- Горилогч энэ ажлыг тус сургуулиас боловсролын зэрэг авахаар бүхэлд нь буюу голлон хийсэн болно.
- Энэ ажлын аль нэг хэсгийг тус сургуульд эсвэл өөр байгууллагад боловсролын зэрэг, мэргэшил авахаар өмнө нь илгээсэн бол түүнийгээ тодорхой заасан болно.
- Бусад хүмүүсийн хэвлүүлсэн ажлаас зөвлөгөө авсан бол түүнийгээ үндэслэсэн болно.
- Бусад хүмүүсийн ажлаас ишлэл хийхдээ эх үүсвэрийг нь заасан болно.
- Миний ажилд тусалсан голлох бүх эх үүсвэрт талархаж байна.
- Ажлыг бусадтай хамтарсан бол алийг нь бусад хүмүүс хийсэн болохыг тодорхой заасан болно.

Гарын үсэг: _____

Огноо: _____

Шинжлэх Ухаан, Технологийн Их Сургууль
Мэдээлэл, Холбооны Технологийн Сургууль

# Хураангуй

## Үүлэн орчинд суурилсан спорт заалны цаг захиалгын 3 давхаргат системийн загварчлал, хэрэгжилт

М.Түвшинжаргал

***Түлхүүр үгс:*** *Flutter, Firebase, Node.js, Docker, AWS, цаг захиалга, спорт заал, үүлэн орчин*

Өнөөгийн дижитал хөгжлийн эрин үед иргэдэд үйлчилгээг хурдан, хялбар, цахимаар хүргэх нь улам бүр чухал болж байна. Өмнөговь аймгийн Даланзадгад хотод таван спорт заал үйл ажиллагаа явуулж байгаа ч иргэд цаг захиалахдаа утсаар залгах, биечлэн очих зэрэг уламжлалт аргыг ашигладаг бөгөөд энэ нь цаг хугацаа дэмий үрэх болон буруу ойлголцол үүсгэх эрсдэлтэй.

Энэхүү бакалаврын төгсөлтийн ажлын зорилго нь Өмнөговь аймгийн Даланзадгад хотын спорт заалнуудын цаг захиалгыг цахимжуулах мобайл аппликейшн хөгжүүлэх явдал юм. Уг апп нь **Flutter** фреймворк болон **Dart** програмчлалын хэлийг ашиглан хөгжүүлэгдсэн бөгөөд Android болон iOS платформд нэгэн зэрэг ажиллах чадвартай. Хэрэглэгчийн интерфэйс нь **Material Design 3** стандартад нийцсэн, говийн цөлийн дүр төрхийг илэрхийлсэн харанхуй өнгөний схем бүхий дизайнтай (цөлийн хар #120D06, алтан элс #F0B429, нарны жаргалтын улбар #D4622A).

Системийн архитектур нь Flutter клиент, **Node.js/Express** бэкэнд (AWS EC2, Токио бүс), **Firebase** (Authentication, Cloud Firestore, Cloud Functions) болон **AI чатботын** үйлчилгээнээс бүрдэнэ. Бэкэнд сервер нь **Docker** контейнержуулалт ашиглан **AWS EC2** дээр байрладаг бөгөөд **GitHub Actions** CI/CD хоолойгоор автоматаар байршуулагдана.

Системийн үндсэн функцүүдэд дараахи зүйлс орно: таван спорт заалны цагийн хуваарь харах, OTP нэвтрэлт (утасны дугаараар), цаг захиалах болон цуцлах, захиалгын түүх харах, **flutter_local_notifications** ашиглан нэг цагийн өмнө сануулга авах, AI чатботтой харилцах. Захиалгын мэдээллийг **Cloud Firestore**-ийн `bookings`, `users`, `fixed_bookings` гэсэн гурван цуглуулгад хадгалдаг. Байгууллагуудын тогтмол (давтагдах) захиалгыг `fixed_bookings` цуглуулгад тусад нь хадгалж, долоо хоногийн тогтмол хуваарийг хэрэгжүүлнэ. Захиалгын код нь **DBK-xxxxx** форматтай.

AI чатботын үйлчилгээ нь **Anthropic Claude Haiku 4.5**-г Firebase Cloud Functions дамжуулан (үндсэн горим) болон **Groq Llama 3.1-8b**-г Express бэкэнд дамжуулан (нөөц горим) ажилладаг. Бэкэнд дэх cron ажил нь 15 минут тутамд дуусгавар болсон захиалгыг автоматаар `completed` төлөвт шилжүүлдэг.

Ажлын явцад шаардлагын дүн шинжилгээ, системийн архитектур зохион байгуулалт, өгөгдлийн загвар боловсруулалт, Flutter апп болон Node.js бэкэнд хэрэгжүүлэлт, Docker/AWS байршуулалт, CI/CD автоматжуулалт болон туршилтын үе шатуудыг дамжсан. Гарсан үр дүнгээс харахад уг мобайл апп нь иргэдийн спорт заал захиалах үйл явцыг ихээхэн хялбарчилж, захиалга хийх хугацааг мэдэгдэхүйц бууруулна гэж тооцоолж байна.

***Keywords:*** *Flutter, Firebase, Node.js, Docker, AWS EC2, GitHub Actions, AI chatbot, mobile application, time booking, sports hall, Dalanzadgad*

The aim of this bachelor's thesis is to develop a mobile application for booking sports hall time slots in Dalanzadgad city, Ömnögovi Province of Mongolia. The application is built using the Flutter framework and Dart language, featuring a dark desert-themed Material Design 3 interface. The system architecture consists of a Flutter mobile client, a Node.js/Express backend deployed on AWS EC2 (Tokyo region) via Docker containers, Firebase services (Authentication, Firestore, Cloud Functions), and AI chat capabilities powered by Anthropic Claude Haiku 4.5 and Groq Llama 3.1-8b. The project employs GitHub Actions for CI/CD automation, AWS ECR for Docker image registry, and flutter_local_notifications for local booking reminders. Five sports venues are supported, with three Firestore collections (bookings, users, fixed_bookings) managing all reservation data.

# Талархал

Энэхүү бакалаврын төгсөлтийн ажлыг гүйцэтгэхэд чиглэл заагч, зөвлөгөө өгч, тасралтгүй дэмжиж ирсэн удирдагч багш **Доктор (Ph.D) профессор А.Алтангэрэл** багш танд гүн талархал илэрхийлье.

Дөрвөн жилийн турш мэдлэг, чадвар, судалгааны арга барилыг эзэмшихэд тусалсан **Мэдээлэл, Холбооны Технологийн Сургуулийн** бүх багш нарт талархал илэрхийлж байна.

Мөн судалгааны явцад туршилтад оролцож, санал хүсэлтээ өгсөн **Өмнөговь аймгийн Даланзадгад хотын спорт заалнуудын** захиралууд болон ажилчдад талархал илэрхийлье.

Эцэст нь, тасралтгүй урам өгч, дэмжиж ирсэн гэр бүлийнхэндээ болон хамт сурдаг нөхдөдөө чин сэтгэлийн талархлаа илэрхийлье.

# Товчилсон үгс

| | |
|---|---|
| UI | User Interface (Хэрэглэгчийн интерфэйс) |
| UX | User Experience (Хэрэглэгчийн туршлага) |
| API | Application Programming Interface (Аппликейшн програмчлалын интерфэйс) |
| SDK | Software Development Kit (Програм хангамжийн хөгжүүлэлтийн хэрэгсэл) |
| IDE | Integrated Development Environment (Нэгдсэн хөгжүүлэлтийн орчин) |
| OS | Operating System (Үйлдлийн систем) |
| iOS | iPhone Operating System (Apple-ийн мобайл үйлдлийн систем) |
| APK | Android Package Kit (Android апп-ын суулгац файл) |
| HTTP | HyperText Transfer Protocol |
| HTTPS | HyperText Transfer Protocol Secure |
| JSON | JavaScript Object Notation (Өгөгдөл солилцооны формат) |
| JWT | JSON Web Token (Нэвтрэлтийн токен формат) |
| REST | Representational State Transfer |
| MVC | Model-View-Controller (Загвар-Харагдац-Удирдагч) |
| DB | Database (Өгөгдлийн сан) |
| SQL | Structured Query Language |
| NoSQL | Not only SQL (Уламжлалт бус өгөгдлийн сан) |
| OTP | One-Time Password (Нэг удаагийн нууц үг) |
| FCM | Firebase Cloud Messaging (Firebase-ийн мэдэгдлийн үйлчилгээ) |
| CI/CD | Continuous Integration / Continuous Deployment (Тасралтгүй нэгтгэлт / байршуулалт) |
| AWS | Amazon Web Services (Амазоны үүлэн үйлчилгээ) |
| EC2 | Elastic Compute Cloud (AWS-ийн виртуал сервер үйлчилгээ) |
| ECR | Elastic Container Registry (AWS-ийн Docker дүрс хадгалах сан) |
| VPC | Virtual Private Cloud (Виртуал хувийн сүлжээ) |
| IAM | Identity and Access Management (AWS нэвтрэлт, эрх удирдлага) |
| AI | Artificial Intelligence (Хиймэл оюун ухаан) |
| LLM | Large Language Model (Том хэлний загвар) |
| SaaS | Software as a Service (Үйлчилгээ хэлбэрийн програм хангамж) |
| BaaS | Backend as a Service (Үйлчилгээ хэлбэрийн бэкэнд) |
| UML | Unified Modeling Language (Нэгдсэн загварчлалын хэл) |
| QR | Quick Response (Шуурхай хариу кодчилол) |

# Гарчиг

**Зохиогчийн эрх хамгаалал** i

**Хураангуй** ii

**Талархал** iv

**Товчилсон үгс** v

**Удирттал** x

**1 ОНОЛЫН ҮНДЭС БА ТЕХНОЛОГИЙН СУДАЛГАА** 1

1.1 2.1 Мобайл апп хөгжүүлэлтийн орчин үеийн арга хандлага . . . . . 1
1.2 2.2 Flutter фреймворк ба Dart хэл . . . . . 1
1.2.1 2.2.1 Flutter-ийн тухай . . . . . 1
1.2.2 2.2.2 Dart програмчлалын хэл . . . . . 1
1.2.3 2.2.3 StatefulWidget ба setState-д суурилсан төлвийн удирдлага 2
1.3 2.3 Firebase бэкэнд үйлчилгээ . . . . . 3
1.3.1 2.3.1 Cloud Firestore . . . . . 3
1.3.2 2.3.2 Firebase Authentication . . . . . 3
1.3.3 2.3.3 Firebase Cloud Functions . . . . . 3
1.4 2.4 Node.js болон Express.js бэкэнд . . . . . 3
1.4.1 2.4.1 Node.js . . . . . 3
1.4.2 2.4.2 Express.js фреймворк . . . . . 4
1.4.3 2.4.3 REST API архитектур . . . . . 4
1.5 2.5 Docker контейнержуулалт . . . . . 4
1.6 2.6 AWS дэд бүтэц . . . . . 5
1.6.1 2.6.1 Amazon EC2 . . . . . 5
1.6.2 2.6.2 Amazon ECR . . . . . 5
1.7 2.7 GitHub Actions CI/CD . . . . . 5
1.8 2.8 AI үйлчилгээнүүд . . . . . 6
1.8.1 2.8.1 Anthropic Claude Haiku . . . . . 6
1.8.2 2.8.2 Groq Llama 3.1-8b . . . . . 6
1.9 2.9 flutter_local_notifications . . . . . 6
1.10 2.10 Ижил төстэй системүүдийн дүн шинжилгээ . . . . . 6

**2 СИСТЕМИЙН ЗОХИОН БАЙГУУЛАЛТ БА ХЭРЭГЖҮҮЛЭЛТ** 8

2.1 3.1 Системийн шаардлагын дүн шинжилгээ . . . . . 8
2.1.1 3.1.1 Функциональ шаардлага . . . . . 8
2.1.2 3.1.2 Функциональ бус шаардлага . . . . . 8
2.2 3.2 Системийн UML диаграммууд . . . . . 8
2.2.1 3.2.1 Хэрэглэгчийн тохиолдлын диаграмм (Use Case Diagram) 8
2.2.2 3.2.2 Цаг захиалах үйл ажиллагааны диаграмм (Activity Diagram) 9
2.2.3 3.2.3 Захиалга үүсгэх дараалын диаграмм (Sequence Diagram) 9
2.2.4 3.2.4 Firestore өгөгдлийн сангийн схем (ER Диаграмм) . . . . 9

2.3 3.3 Системийн архитектур . . . 9
2.3.1 3.3.1 Ерөнхий архитектур . . . 9
2.3.2 3.3.2 Flutter апп-ын архитектур . . . 10
2.3.3 3.3.3 API URL-ийн нэг эх сурвалж (lib/config.dart) . . . 10
2.4 3.4 Өгөгдлийн сангийн зохион байгуулалт . . . 11
2.4.1 3.4.1 `users` collection . . . 11
2.4.2 3.4.2 `bookings` collection . . . 12
2.4.3 3.4.3 `fixed_bookings` collection . . . 12
2.4.4 3.4.4 Hardcoded спорт заалнуудын жагсаалт . . . 13
2.5 3.5 Flutter апп-ын хэрэгжүүлэлт . . . 13
2.5.1 3.5.1 Харанхуй говийн дизайн (Dark Desert Theme) . . . 13
2.5.2 3.5.2 4 tab-тай доод навигаци . . . 14
2.5.3 3.5.3 Утасны OTP нэвтрэлт . . . 15
2.5.4 3.5.4 Нүүр дэлгэц — Спорт заалнуудын жагсаалт . . . 15
2.5.5 3.5.5 Цагийн хуваарь ба захиалгын дэлгэц . . . 15
2.5.6 3.5.6 Локал мэдэгдлийн хэрэгжүүлэлт . . . 17
2.6 3.6 Node.js/Express бэкэндийн хэрэгжүүлэлт . . . 17
2.6.1 3.6.1 Бэкэндийн бүтэц . . . 17
2.6.2 3.6.2 Үндсэн API endpoint-ууд . . . 17
2.6.3 3.6.3 Cron ажил — Дуусгавар болсон захиалгыг автоматаар шинэчлэх . . . 18
2.7 3.7 AI чатботын хэрэгжүүлэлт . . . 19
2.7.1 3.7.1 Архитектур — Үндсэн болон нөөц горим . . . 19
2.7.2 3.7.2 Firebase Cloud Functions — Claude Haiku . . . 19
2.7.3 3.7.3 Express бэкэнд — Groq Llama нөөц горим . . . 19
2.8 3.8 Docker контейнержуулалт ба AWS байршуулалт . . . 20
2.8.1 3.8.1 Docker Image . . . 20
2.8.2 3.8.2 GitHub Actions CI/CD Workflow . . . 20
2.9 3.9 Туршилт ба үр дүнгийн үнэлгээ . . . 21
2.9.1 3.9.1 Функциональ туршилт . . . 21
2.9.2 3.9.2 Гүйцэтгэлийн хэмжилт . . . 22
2.9.3 3.9.3 CI/CD хоолойн туршилт . . . 22

**3 СИСТЕМИЙН ТУРШИЛТ БА ҮНЭЛГЭЭ 24**
3.1 4.1 Туршилтын аргачлал . . . 24
3.2 4.2 Функциональ туршилт . . . 24
3.2.1 4.2.1 Туршилтын тохиолдлуудын хүснэгт . . . 24
3.2.2 4.2.2 Тогтмол захиалгын (Fixed Booking) туршилт . . . 25
3.3 4.3 Гүйцэтгэлийн туршилт . . . 25
3.3.1 4.3.1 Хариу цагийн хэмжилт . . . 25
3.3.2 4.3.2 Cron ажлын туршилт . . . 25
3.4 4.4 Нэгтгэлийн туршилт . . . 26
3.5 4.5 Хэрэглэхэд хялбар байдлын үнэлгээ . . . 26
3.5.1 4.5.1 Хэрэглэгчийн туршилт . . . 26
3.5.2 4.5.2 UI/UX үнэлгээ . . . 27
3.6 4.6 Аюулгүй байдлын шалгалт . . . 27
3.7 4.7 Туршилтын нэгтгэсэн дүн . . . 28

**Дүгнэлт 29**

**Ном зүй 31**

# Зургийн жагсаалт

2.1 Хэрэглэгчийн тохиолдлын диаграмм . . . . . 9
2.2 Цаг захиалах үйл ажиллагааны диаграмм . . . . . 10
2.3 Захиалга үүсгэх дараалын диаграмм . . . . . 11
2.4 Firestore өгөгдлийн сангийн схем . . . . . 12
2.5 Системийн ерөнхий архитектурын диаграмм . . . . . 13

# Хүснэгтийн жагсаалт

1.1 Цаг захиалгын системүүдийн харьцуулалт . . . . . 7

2.1 Функциональ туршилтын үр дүн . . . . . 22

3.1 Функциональ туршилтын тохиолдлууд . . . . . 24
3.2 Системийн хариу цагийн хэмжилт . . . . . 26
3.3 Нэгтгэлийн туршилтын үр дүн . . . . . 26
3.4 Хэрэглэхэд хялбар байдлын үнэлгээний үр дүн . . . . . 27
3.5 UI/UX үнэлгээний дундаж оноо (1–5 шкал) . . . . . 27
3.6 Туршилтын нийт үр дүн . . . . . 28

# УДИРТТАЛ

## Сэдэв сонгосон үндэслэл

Мэдээллийн технологийн хурдацтай хөгжил нь орчин үеийн иргэдийн өдөр тутмын амьдралыг ихээхэн өөрчилж байна. Цахим захиалга, онлайн үйлчилгээ нь дэлхийн өнцөг булан бүрт нэвтэрч, хүмүүсийн цаг хугацааг хэмнэж, үйлчилгээний чанарыг дээшлүүлж байна. Гэвч Монгол улсын орон нутгийн хотуудад, тухайлбал Өмнөговь аймгийн Даланзадгад хотод, уламжлалт үйлчилгээний хэлбэр давамгайлсаар байна.

Даланзадгад хотод өнөөдрийн байдлаар хэд хэдэн спорт заал, тамирын цогцолбор үйл ажиллагаа явуулж байгаа боловч иргэд цаг захиалахдаа дараах бэрхшээлтэй тулгардаг:

- Утсаар залгаж цаг захиалах үед шугам завгүй байх, буруу ойлголцол үүсэх;
- Биечлэн очиж захиалахад цаг, зардал зарцуулагдах;
- Боломжтой цагийн мэдээлэл тодорхойгүй, ил тод бус байх;
- Захиалгаа цуцлах, өөрчлөх боломж хязгаарлагдмал байх;
- Захиалгын баталгаажуулалт, сануулга авах боломжгүй байх.

Эдгээр асуудлыг шийдвэрлэхийн тулд орчин үеийн мобайл технологийг ашиглан цаг захиалгын систем хөгжүүлэх нь чухал шаардлага болж байна. **Flutter** фреймворк нь нэг удаа бичиж Android болон iOS платформд нэгэн зэрэг гаргах боломж олгодог тул хөгжүүлэлтийн зардал, хугацааг мэдэгдэхүйц бууруулдаг. **Firebase** бэкэнд үйлчилгээ нь бодит цагийн өгөгдөл солилцоо, найдвартай нэвтрэлт, тасралтгүй ажиллагааг хангадаг. Харин **Node.js/Express** бэкэнд, **Docker** контейнержуулалт, **AWS** дэд бүтэц нь системийг үүлэн орчинд байршуулж, тасралтгүй хүргэх боломжийг олгодог.

## Зорилго

**Зорилго:** Өмнөговь аймгийн Даланзадгад хотын спорт заалнуудын цаг захиалгыг цахимжуулах, хэрэглэгчдэд хялбар, хурдан, ил тод байдлаар захиалга хийх боломж олгох **Flutter** мобайл аппликейшн болон **Node.js/Express** бэкэнд бүхий 3 давхаргат системийг **үүлэн орчинд** (AWS) хөгжүүлж, байршуулах.

## Зорилт

Уг зорилгод хүрэхийн тулд дараах зорилтуудыг дэвшүүлж байна. Үүнд:

1. Даланзадгад хотын спорт заалнуудын цаг захиалгын одоогийн нөхцөл байдал, иргэдийн хэрэгцээг судлах;
2. Цаг захиалгын мобайл апп-ын шаардлагын дүн шинжилгээ хийх, системийн загварчлал боловсруулах;
3. Flutter болон Firebase технологийг ашиглан 3 давхаргат системийн архитектур боловсруулах;
4. Хэрэглэгчийн UI/UX загвар гаргаж, мобайл аппликейшн хэрэгжүүлэх;

5. Node.js/Express бэкэнд хөгжүүлж, Docker ашиглан AWS EC2-д байршуулах;
6. GitHub Actions CI/CD хоолойгоор автоматжуулалт хэрэгжүүлэх;
7. Системийн гүйцэтгэл, хэрэглэгчийн сэтгэл ханамжийг туршиж, үнэлэх.

### Судлагдсан байдал

Спорт заал болон үйлчилгээний цаг захиалгын мобайл аппликейшн хөгжүүлэлтийн чиглэлд олон улсын болон дотоодын судлаачид тодорхой хэмжээний ажил хийсэн байна.

Олон улсын түвшинд **Mindbody** (АНУ), **Vagaro** (АНУ) зэрэг цаг захиалгын платформууд спорт болон фитнесийн салбарт өргөн хэрэглэгдэж байна. Эдгээр систем нь iOS болон Android-д ажилладаг хэдий ч дотоодын зах зээлд, ялангуяа орон нутгийн хэрэгцээнд тохиромжтой нутагшсан шийдэл биш юм.

Монголын хэмжээнд Амарсайханы Билгүүн [1] мобайл аппликейшн хөгжүүлэлтийн аргачлалын асуудлыг судалсан бол Т.Амартүвшин, Б.Зоригтбаатар нар [2] мобайл апп хөгжүүлэлтийн онолын үндсийг тодорхойлсон байна. Гэхдээ тухайлан **үүлэн орчинд суурилсан**, **3 давхаргат архитектур** бүхий спорт заалны цаг захиалгын тусгай системийн талаарх судалгаа орон нутгийн хэмжээнд хийгдээгүй байна.

Иймд энэхүү ажил нь орон нутгийн хэрэгцээнд нийцсэн, орчин үеийн технологийн бүрэн стек ашигласан цоо шинэ шийдлийг санал болгож байна.

### Шинэлэг тал

Энэхүү ажлын шинэлэг тал нь дараахид оршино:

1. **3 давхаргат архитектур:** Flutter клиент — Node.js/Express бэкэнд — Firebase/AWS дэд бүтэц гэсэн тодорхой давхаргаар системийг зохион байгуулсан;
2. **Үүлэн орчин:** Docker контейнержуулалт болон AWS EC2 дэд бүтцийг ашиглан тасралтгүй, хэмжээгээр уян хатан системийг бий болгосон;
3. **CI/CD автоматжуулалт:** GitHub Actions ашиглан код шинэчлэгдэх бүрт автоматаар байршуулдаг хоолой тавьсан;
4. **AI чатбот:** Anthropic Claude Haiku болон Groq Llama загваруудыг нөөцлөлтийн горимоор нэгтгэж, хэрэглэгчийг туслах ухаалаг системийг хэрэгжүүлсэн;
5. **Орон нутгийн хэрэгцээ:** Даланзадгад хотын таван спорт заалд тохирсон байгууллагуудын тогтмол захиалга (`fixed_bookings`) зэрэг тусгай функцийг хэрэгжүүлсэн.

### Судалгааны хамрах хүрээ

**Хамрах хүрээ:** Энэхүү ажлын хүрээнд дараахи зүйлс орно:

- Иргэдэд зориулсан мобайл апп (Android болон iOS);
- Firebase дээр суурилсан нэвтрэлт (OTP), өгөгдлийн сан;
- Node.js/Express REST API бэкэнд;

- Docker контейнержуулалт, AWS EC2/ECR байршуулалт;
- GitHub Actions CI/CD автоматжуулалт;
- AI чатботын үйлчилгээ (Claude Haiku, Groq Llama);
- Локал мэдэгдлийн систем (flutter_local_notifications).

Уг ажилд төлбөрийн систем нэгтгэх болон GPS байршлын нарийн функцүүд хамрагдахгүй болно.

### Ач холбогдол

**Практик ач холбогдол:** Даланзадгад хотын иргэдэд спорт заал захиалах үйл явцыг хялбарчилж, цаг хугацааг хэмнэнэ. Спорт заалны ажилтнуудад захиалгын менежментийг автоматжуулж, ажлын ачааллыг бууруулна.

**Шинжлэх ухааны ач холбогдол:** Flutter болон Firebase технологийг Node.js/Express REST API, Docker/AWS дэд бүтэц, GitHub Actions CI/CD автоматжуулалт, AI чатбот үйлчилгээтэй хосолсон орчин үеийн бүрэн стекийн туршлагыг Монголын орон нутгийн нөхцөлд хэрэглэж буй анхны бүтээлүүдийн нэг болно.

### Ажлын бүтэц

Энэхүү бакалаврын төгсөлтийн ажил дараахи бүтэцтэй:

**Удирттал:** Судалгааны үндэслэл, зорилго, зорилт, судлагдсан байдал, шинэлэг тал, хамрах хүрээ, ач холбогдлыг тодорхойлсон.

**1-р бүлэг — Онолын үндэс ба технологийн судалгаа:** Мобайл апп хөгжүүлэлтийн орчин үеийн арга хандлага, Flutter фреймворк, Firebase үйлчилгээ, Node.js/Express, Docker, AWS, GitHub Actions, AI технологиудын онолын үндсийг судалсан. Ижил төстэй системүүдтэй харьцуулсан.

**2-р бүлэг — Системийн шаардлага, загварчлал:** Шаардлагын дүн шинжилгээ, системийн 3 давхаргат архитектур, өгөгдлийн сангийн загвар, UML диаграммуудыг тусгасан.

**3-р бүлэг — Системийн зохион байгуулалт ба хэрэгжүүлэлт:** Flutter апп-ын хэрэгжүүлэлт, Node.js бэкэнд, Docker/AWS байршуулалт, CI/CD автоматжуулалтыг тусгасан.

**4-р бүлэг — Системийн туршилт ба үнэлгээ:** Функциональ туршилт, гүйцэтгэлийн хэмжилт, нэгтгэлийн туршилт, хэрэглэхэд хялбар байдлын үнэлгээний үр дүнг тусгасан.

**Дүгнэлт:** Ажлын үр дүн, цаашдын хөгжлийн чиглэлийг дүгнэсэн.

БҮЛЭГ 1

# ОНОЛЫН ҮНДЭС БА ТЕХНОЛОГИЙН СУДАЛГАА

## 1.1 2.1 Мобайл апп хөгжүүлэлтийн орчин үеийн арга хандлага

Мобайл аппликейшн хөгжүүлэлтэд өнөөдөр гурван үндсэн арга хандлага байдаг: **Native**, **Hybrid**, **Cross-platform**. Эдгээрийг харьцуулбал:

- **Native апп** — iOS-д Swift/Objective-C, Android-д Kotlin/Java ашигладаг. Гүйцэтгэл хамгийн өндөр боловч платформ бүрт тусдаа код бичих шаардлагатай.
- **Hybrid апп** — HTML, CSS, JavaScript ашиглан WebView дотор ажиллана. Хөгжүүлэлт хялбар боловч гүйцэтгэл доогуур.
- **Cross-platform апп** — Нэг кодоос олон платформд гаргана. Flutter, React Native зэрэг хэрэгсэл ашигладаг.

Энэхүү ажилд cross-platform хандлагыг сонгосон бөгөөд үүний тулд **Flutter** фреймворкийг ашигласан. Flutter нь Google-ийн боловсруулсан, Dart хэл дээр суурилсан, Android болон iOS платформд нэгэн зэрэг ажиллах чадвартай, гүйцэтгэлийн хувьд native-д ойр шийдэл юм [3]. Cross-platform хандлагыг сонгосноор нэг кодын баазаас хоёр платформ дэмжих боломжтой бөгөөд хөгжүүлэлтийн нийт хугацаа, зардлыг мэдэгдэхүйц бууруулдаг.

## 1.2 2.2 Flutter фреймворк ба Dart хэл

### 1.2.1 2.2.1 Flutter-ийн тухай

**Flutter** нь Google компанийн 2018 онд гаргасан нээлттэй эхийн UI фреймворк юм. Flutter-ийн гол давуу талд дараахи зүйлс орно:

- **Hot Reload** — Кодын өөрчлөлтийг апп-г дахин эхлүүлэхгүйгээр шууд харах боломж;
- **Widget-д суурилсан архитектур** — UI бүрэлдэхүүн хэсэг болгоныг widget хэлбэрээр тодорхойлно;
- **Skia/Impeller render engine** — Бүх UI-г платформаас үл хамааран өөрөө зурдаг тул платформ бүрт ижил харагдах баталгаа олгодог;
- **Нэг кодын бааз** — Android, iOS, Web, Desktop платформд нэгэн зэрэг гаргах боломж.

Flutter нь version 3.x хүрэхдээ Material Design 3 (Material You) стандартыг бүрэн дэмждэг болж, `ThemeData` классаар дамжуулан дүрс, өнгө, хэлбэр бүх зүйлийг системтэй тохируулах боломж нээгдсэн [4]. Энэхүү ажилд Flutter 3.x хувилбарыг ашигласан.

### 1.2.2 2.2.2 Dart програмчлалын хэл

**Dart** нь Google-ийн боловсруулсан, object-oriented, strongly typed хэл юм. Flutter хөгжүүлэлтэд Dart ашигладаг бөгөөд дараахи онцлогтой [5]:

- **Async/Await** — Асинхрон үйлдлийг хялбараар бичих, Future болон Stream дэмждэг;

- **Null Safety** — Null pointer алдааг compile time-д илрүүлэх;
- **AOT ба JIT compile** — Хурдан хөгжүүлэлт (JIT) болон хурдан гүйцэтгэл (AOT) нэгэн зэрэг хангах;
- **Stream** — Бодит цагийн өгөгдөл боловсруулах, Firebase Firestore-ийн real-time listener-тэй холбоход тохиромжтой.

Dart хэлний `class` бүтэц нь Flutter-ийн widget системтэй нягт уялддаг. `StatelessWidget` нь өөрчлөлтгүй UI-г илэрхийлж, `StatefulWidget` нь дотоод `State` объектоор дамжуулан өөрчлөгддөг UI-г удирддаг.

### 1.2.3 2.2.3 StatefulWidget ба setState-д суурилсан төлвийн удирдлага

Flutter апп-д төлвийн удирдлагыг (state management) зөв хийх нь апп-ын найдвартай байдал, гүйцэтгэлд шууд нөлөөлнө. Энэ ажилд **StatefulWidget** болон **setState** хослолыг ашигласан. Энэ нь Flutter-ийн бүтэц дэх хамгийн анхдагч, ойлгомжтой механизм бөгөөд нэмэлт багц шаарддаггүй.

Provider, Riverpod зэрэг өндөр түвшний төлвийн удирдлагын хэрэгслүүдтэй харьцуулахад `setState` нь дараахи тохиолдолд тохиромжтой:

- Апп-ын нарийн төвөгтэй байдал тодорхой хэмжээнд байх;
- Шинэ хөгжүүлэгчдэд ойлгомжтой, уншиж болохуйц байх шаардлагатай;
- Нэмэлт dependency нэмэхгүйгээр апп-ын хэмжээг жижиг байлгах.

Эх код 1.1: StatefulWidget ба setState жишээ

```
class BookingScreen extends StatefulWidget {
  const BookingScreen({super.key});
  @override
  State<BookingScreen> createState() => _BookingScreenState();
}

class _BookingScreenState extends State<BookingScreen> {
  String? selectedTimeSlot;
  bool isLoading = false;

  void _selectSlot(String slot) {
    setState(() {
      selectedTimeSlot = slot;
    });
  }
  // ...
}
```

Апп-ын ерөнхий навигацийг `BottomNavigationBar` widget дамжуулан 4 tab-тай хэрэгжүүлсэн: Нүүр (Home), Захиалгууд (Bookings), Чат (Chat), Профайл (Profile). Идэвхтэй tab-ийн индексийг root StatefulWidget-ийн `setState`-д хадгалдаг.

## 1.3 2.3 Firebase бэкэнд үйлчилгээ

**Firebase** нь Google-ийн санал болгодог BaaS (Backend as a Service) платформ юм. Уг платформ нь мобайл апп хөгжүүлэгчдэд бэкэнд дэд бүтэц тохируулах шаардлагагүйгээр өгөгдлийн сан, нэвтрэлт зэрэг үйлчилгээг ашиглах боломж олгодог [6].

### 1.3.1 2.3.1 Cloud Firestore

**Cloud Firestore** нь NoSQL, баримт жишгийн (document-oriented) өгөгдлийн сан юм [7]. Бодит цагийн шинэчлэлт (real-time listeners), өгөгдлийг offline хадгалах, автомат масштабчлал зэрэг онцлогтой. Firestore нь **Collection** болон **Document** бүтэцтэй:

- **Collection** — Баримтуудын цуглуулга (энэ ажилд: `users`, `bookings`, `fixed_bookings`);
- **Document** — Талбаруудын цуглуулга, JSON форматтай төстэй.

Firestore-ийн security rules нь collection бүрт хэрэглэгч хандах эрхийг нарийн тохируулах боломж олгодог. Энэ ажилд нэвтрэлт хийсэн хэрэглэгч зөвхөн өөрийн захиалгыг унших, шинэ захиалга бичих эрхтэй.

### 1.3.2 2.3.2 Firebase Authentication

**Firebase Authentication** нь хэрэглэгчийн нэвтрэлтийг аюулгүй удирдах үйлчилгээ юм [8]. Энэ ажилд зөвхөн **утасны дугаар OTP** нэвтрэлтийг ашигласан. Хэрэглэгч утасны дугаараа оруулснаар Firebase нь SMS OTP илгээж, хэрэглэгч кодоо оруулсны дараа `PhoneAuthCredential` үүсгэн нэвтрүүлдэг.

Утасны дугааргүй, имэйл/нууц үгийн нэвтрэлт болон нийгмийн сүлжээний нэвтрэлтийг энэ ажилд ашиглаагүй. Энэ нь Монгол хэрэглэгчийн хувьд утасны дугаар бол хамгийн тохиромжтой, хурдан нэвтрэлтийн арга болохтой холбоотой.

### 1.3.3 2.3.3 Firebase Cloud Functions

**Firebase Cloud Functions** нь серверлэс (serverless) тооцооллын үйлчилгээ юм. Энэ ажилд Cloud Functions-ийг AI чатботын үндсэн endpoint болгон ашигласан: Flutter апп Cloud Function руу хүсэлт илгээхэд уг функц Anthropic-ийн API руу дамжуулан Claude Haiku хариулт авч буцаана. Cloud Functions нь серверийн дэд бүтцийг өөрөө удирддаг тул хөгжүүлэгч апп-ын логикт төвлөрөх боломжтой болдог.

## 1.4 2.4 Node.js болон Express.js бэкэнд

### 1.4.1 2.4.1 Node.js

**Node.js** нь Chrome V8 JavaScript engine дээр суурилсан серверийн талын JavaScript ажиллуулах орчин юм [9]. Node.js-ийн гол давуу талууд:

- **Event-driven, non-blocking I/O** — Олон нэгэн зэрэг холболтыг цөөн нөөцөөр боловсруулах чадвартай;
- **npm экосистем** — Дэлхийн хамгийн том нээлттэй эхийн багцын сан;

- **JavaScript нэгдсэн байдал** — Клиент болон серверийн талд нэг хэл ашиглах боломж;
- **Хурдан хөгжүүлэлт** — JSON боловсруулалт, REST API бүтээхэд маш тохиромжтой.

### 1.4.2 2.4.2 Express.js фреймворк

**Express.js** нь Node.js дээр суурилсан хөнгөн, уян хатан веб фреймворк юм [10]. RESTful API барихад хамгийн өргөн хэрэглэгддэг Node.js фреймворк бөгөөд дараахи онцлогтой:

- **Middleware систем** — Хүсэлт боловсруулах дамжлагуудыг зохион байгуулах;
- **Router** — URL хаягийн чиглүүлэлтийг модуль болгон хуваах;
- **Хөнгөн бүтэц** — Шаардлагатай зүйлийг л ачаалдаг.

Энэ ажлын бэкэнд нь Express.js ашиглан дараахи API endpoint-уудыг хангадаг: `/api/chat` (Groq AI чат), `/api/users` (хэрэглэгчийн мэдээлэл), `/api/bookings` (захиалгын менежмент), `/api/schedule` (цагийн хуваарь).

### 1.4.3 2.4.3 REST API архитектур

**REST** (Representational State Transfer) нь веб үйлчилгээний архитектурын хэв маяг юм. REST API нь HTTP протоколын GET, POST, PUT, DELETE арга ашиглан нөөцийг удирддаг. Энэ ажилд Flutter апп болон Node.js бэкэнд хоорондын харилцааг RESTful API-аар гүйцэтгэдэг бөгөөд өгөгдлийн солилцоог JSON форматаар хийдэг.

## 1.5 2.5 Docker контейнержуулалт

**Docker** нь аппликейшн болон түүний бүх хамаарлыг тусгаарлагдсан контейнерт багцалдаг платформ юм [11]. Docker ашиглахын давуу тал:

- **Тогтвортой орчин** — Хөгжүүлэлтийн болон продакшн орчин яг ижил байх баталгаа;
- **Хурдан байршуулалт** — Контейнер хэдхэн секундэд эхлэнэ;
- **Тусгаарлалт** — Системийн бусад хэсгийг нөлөөлөхгүй;
- **Хэмжээс (Scale)** — Хэрэгцээ нэмэгдэхэд хялбар масштабчлал.

**Dockerfile** нь апп-ын дүрсийг (image) тодорхойлдог тохиргооны файл юм. Express бэкэндийн Dockerfile нь Node.js суурь дүрс дээр `package.json`-г суулгаж, эх кодыг хуулан, 3000 портоор сервер эхлүүлдэг.

Эх код 1.2: Node.js бэкэндийн Dockerfile

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
```

```
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

## 1.6 2.6 AWS дэд бүтэц

### 1.6.1 2.6.1 Amazon EC2

**Amazon EC2** (Elastic Compute Cloud) нь AWS-ийн виртуал сервер (instance) үйлчилгээ юм [12]. Энэ ажилд Node.js/Express бэкэнд сервер нь **ap-northeast-1** (Токио) бүсийн EC2 instance дээр ажилладаг. EC2 нь дараахи давуу талтай:

- Серверийн тохиргоог бүрэн хянах боломж;
- Docker контейнер ажиллуулах дэмжлэг;
- Security Group-ийн тусламжтай сүлжээний хандалтыг нарийн удирдах.

### 1.6.2 2.6.2 Amazon ECR

**Amazon ECR** (Elastic Container Registry) нь AWS-ийн Docker дүрсний бүртгэлийн үйлчилгээ юм [13]. GitHub Actions-аар build хийсэн Docker дүрсийг ECR-д push хийж, EC2 instance тэр дүрсийг татан ажиллуулдаг. Энэ нь дүрсний нийцтэй удирдлага, нууцлалыг хангадаг.

## 1.7 2.7 GitHub Actions CI/CD

**GitHub Actions** нь GitHub платформ дахь CI/CD (Continuous Integration / Continuous Deployment) автоматжуулалтын хэрэгсэл юм [14]. Workflow нь YAML файлаар тодорхойлогддог бөгөөд тодорхой нөхцөл (жишээ: `backend/` хавтасны файл өөрчлөгдөх) биелэхэд автоматаар ажилладаг.

Энэ ажлын CI/CD хоолой нь дараахи алхмуудтай:

1. Хөгжүүлэгч `backend/` хавтаст өөрчлөлт хийж GitHub-д push хийнэ;
2. GitHub Actions trigger ажиллана;
3. Node.js бэкэнд кодоос Docker дүрс build хийнэ;
4. Build хийсэн дүрсийг AWS ECR-д push хийнэ;
5. SSH-аар AWS EC2 instance-д холбогдож ECR-аас шинэ дүрсийг pull хийнэ;
6. Хуучин контейнерийг зогсоож, шинэ контейнерийг ажиллуулна.

GitHub Actions нь AWS-ийн нэвтрэлтийн нууцлал (access key, secret), EC2-ийн SSH түлхүүр зэргийг GitHub Secrets-д хадгалан аюулгүй ашигладаг.

## 1.8 2.8 AI үйлчилгээнүүд

### 1.8.1 2.8.1 Anthropic Claude Haiku

**Anthropic Claude Haiku 4.5** нь Anthropic компанийн хурдан, хэмнэлттэй LLM загвар юм [15]. Claude Haiku нь богино хариулт өгдөг асуудлуудад маш хурдан ажиллах бөгөөд API дамжуулан ашиглах боломжтой. Энэ ажилд Firebase Cloud Functions дотор Anthropic-ийн SDK-г ашиглан спорт заалтай холбоотой асуултуудад хариулдаг AI чатбот хэрэгжүүлсэн. Cloud Functions нь API түлхүүрийг Flutter кодод задруулахгүйгээр аюулгүй хадгалах боломж олгодог.

### 1.8.2 2.8.2 Groq Llama 3.1-8b

**Groq** нь LPU (Language Processing Unit) процессор ашиглан LLM загваруудыг асаар хурдан ажиллуулдаг платформ юм [16]. Groq дээрх **Llama 3.1-8b** загварыг Node.js/Express бэкэнд дамжуулан нөөц горим болгон ашигласан. Хэрэв Firebase Cloud Functions-ийн Claude хариулт удаан болох буюу алдаа гарвал Flutter апп `lib/config.dart`-д тодорхойлсон бэкэнд URL-аар Groq API-руу хандана.

## 1.9 2.9 flutter_local_notifications

**flutter_local_notifications** нь Firebase Cloud Messaging (FCM) ашиглахгүйгээр төхөөрөмж дотор мэдэгдэл (local notification) харуулах Flutter багц юм [17]. Энэ нь серверийн тал байхгүйгээр мэдэгдэл үүсгэх боломжтой бөгөөд захиалгын 1 цагийн өмнө сануулга илгээхэд тохиромжтой:

- Хэрэглэгч цаг захиалах үед Flutter апп тухайн цагаас 1 цаг хасаж сануулгын мэдэгдлийг schedule хийнэ;
- FCM сервер байхгүй тул өгөгдлийн дамжуулалт хялбар, найдвартай;
- Android болон iOS аль алинд дэмжлэгтэй.

## 1.10 2.10 Ижил төстэй системүүдийн дүн шинжилгээ

Дэлхийн болон Монгол дахь цаг захиалгын системүүдийг судалж дараахи дүгнэлтэд хүрсэн:

- **Mindbody, Booksy** (дэлхийн платформ) — Олон функцтэй боловч орон нутгийн нөхцөлд тохируулах боломжгүй, Монгол хэлгүй;
- **Монгол дахь ижил апп-ууд** — Хязгаарлагдмал функц, нэг заалд зориулагдсан, ерөнхий шийдэл дутмаг;
- **Энэ ажлын давуу тал** — Даланзадгад хотын таван заалыг нэг платформд нэгтгэсэн, говийн дүр төрхтэй дизайн, AI чатбот, Node.js+Firebase хосолсон архитектур, Docker/AWS автоматжуулсан байршуулалт бүхий орчин үеийн шийдэл.

Хүснэгт 1.1: Цаг захиалгын системүүдийн харьцуулалт

| **Систем** | **Cross-platform** | **AI чат** | **CI/CD** | **Монгол хэл** |
|---|---|---|---|---|
| Mindbody | Тийм | Үгүй | – | Үгүй |
| Booksy | Тийм | Үгүй | – | Үгүй |
| Орон нутгийн апп | Заримдаа | Үгүй | Үгүй | Тийм |
| Энэ ажил | Тийм | Тийм | Тийм | Тийм |

БҮЛЭГ 2

# СИСТЕМИЙН ЗОХИОН БАЙГУУЛАЛТ БА ХЭРЭГЖҮҮЛЭЛТ

## 2.1 3.1 Системийн шаардлагын дүн шинжилгээ

### 2.1.1 3.1.1 Функциональ шаардлага

Системийн функциональ шаардлагыг иргэн (хэрэглэгч) гэсэн нэг үүрэгт тулгуурлан тодорхойлсон. Уг систем нь **администраторын самбаргүй** бөгөөд захиалгын удирдлага бүхэлдээ хэрэглэгчид болон автомат cron ажлаар явагддаг.

**Хэрэглэгчийн функцүүд:**

1. Утасны дугаараар OTP авч нэвтрэх, бүртгэл үүсгэх;
2. Таван спорт заалны жагсаалтыг нүүр дэлгэцэд харах;
3. Тухайн заалны боломжтой болон хаагдсан цагийн хуваарийг харах;
4. Цаг захиалах (заал, огноо, цагийн үе сонгох), DBK-xxxxx форматын захиалгын код авах;
5. Идэвхтэй захиалгаа цуцлах;
6. Захиалгын түүхийг захиалгын дэлгэцэд харах;
7. Захиалгаас 1 цагийн өмнө локал мэдэгдэл хүлээн авах;
8. AI чатботтой спорт заалтай холбоотой асуулт асуух.

**Системийн автомат функцүүд:**

1. 15 минут тутамд cron ажил дуусгавар болсон захиалгыг `completed` төлөвт шилжүүлнэ;
2. Байгууллагуудын тогтмол (давтагдах) захиалгыг `fixed_bookings` цуглуулгаас уншиж долоо хоног бүр цагийн хуваарьт тусгана.

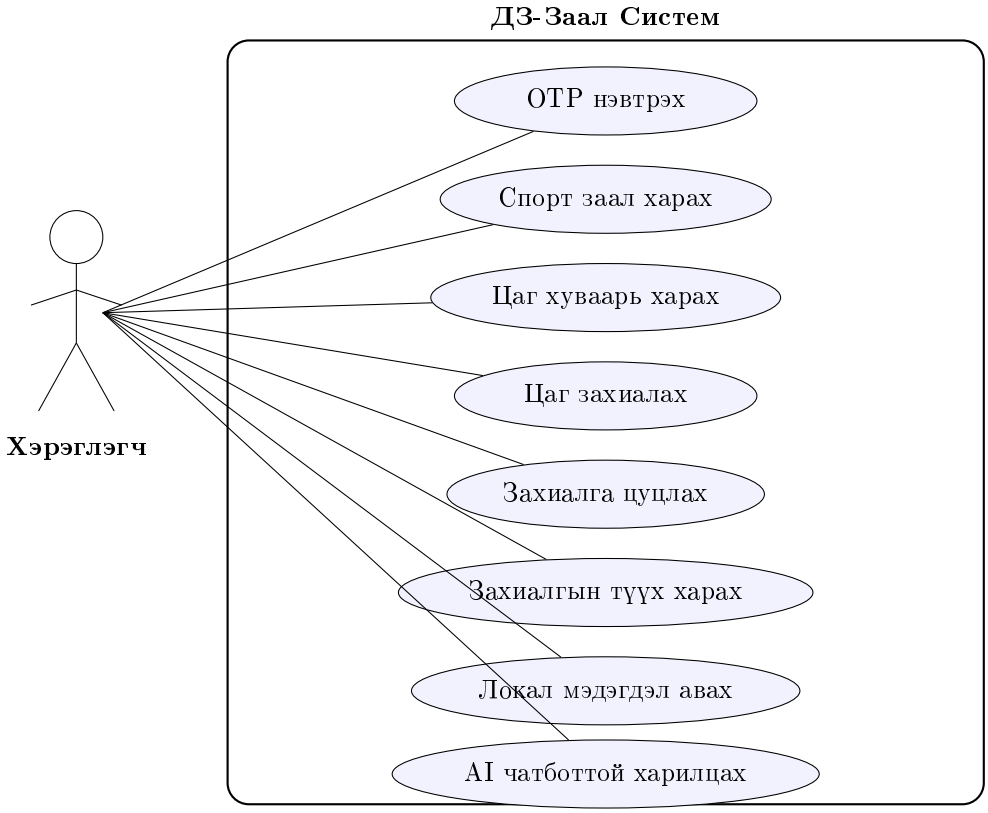
### 2.1.2 3.1.2 Функциональ бус шаардлага

- **Гүйцэтгэл** — Апп нь 3 секундын дотор ачаалах, API хүсэлт 2 секундын дотор хариулах;
- **Найдвартай байдал** — Firebase болон AWS-ийн 99.9% uptime баталгаанд тулгуурлах;
- **Аюулгүй байдал** — Нэвтрэлтийн мэдээллийг Firebase Auth-аар хамгаалах, өгөгдөл дамжуулалтыг HTTPS-ээр шифрлэх, AWS нууцлалыг GitHub Secrets-д хадгалах;
- **Хэрэглэхэд хялбар** — Material Design 3 стандартад нийцсэн, харанхуй горимтай дизайн бүхий UI;
- **Платформ** — Android 8.0+ болон iOS 13.0+ дэмжих.

## 2.2 3.2 Системийн UML диаграммууд

### 2.2.1 3.2.1 Хэрэглэгчийн тохиолдлын диаграмм (Use Case Diagram)

Хэрэглэгчийн тохиолдлын диаграмм нь системийн гадаад оролцогч болон тэдгээрийн гүйцэтгэх үндсэн функцуудыг тодорхойлно. Энэхүү системд **Хэрэглэгч** гэсэн нэг үндсэн actor оролцоно.




Зураг 2.1: Хэрэглэгчийн тохиолдлын диаграмм

### 2.2.2 3.2.2 Цаг захиалах үйл ажиллагааны диаграмм (Activity Diagram)

Цаг захиалах үйл явцын дараалыг үйл ажиллагааны диаграммаар дүрсэлсэн. Хэрэглэгч нэвтэрсний дараа заал, огноо, цагийн үе сонгож захиалга үүсгэнэ. Сонгосон цаг аль хэдийн захиалагдсан бол систем алдаа мэдэгдэж өөр цаг сонгохыг хүснэ.

### 2.2.3 3.2.3 Захиалга үүсгэх дараалын диаграмм (Sequence Diagram)

Захиалга үүсгэх үйл явцын оролцогч тал бүрийн харилцан үйлдлийн дараалыг дараалын диаграммаар илэрхийлсэн.

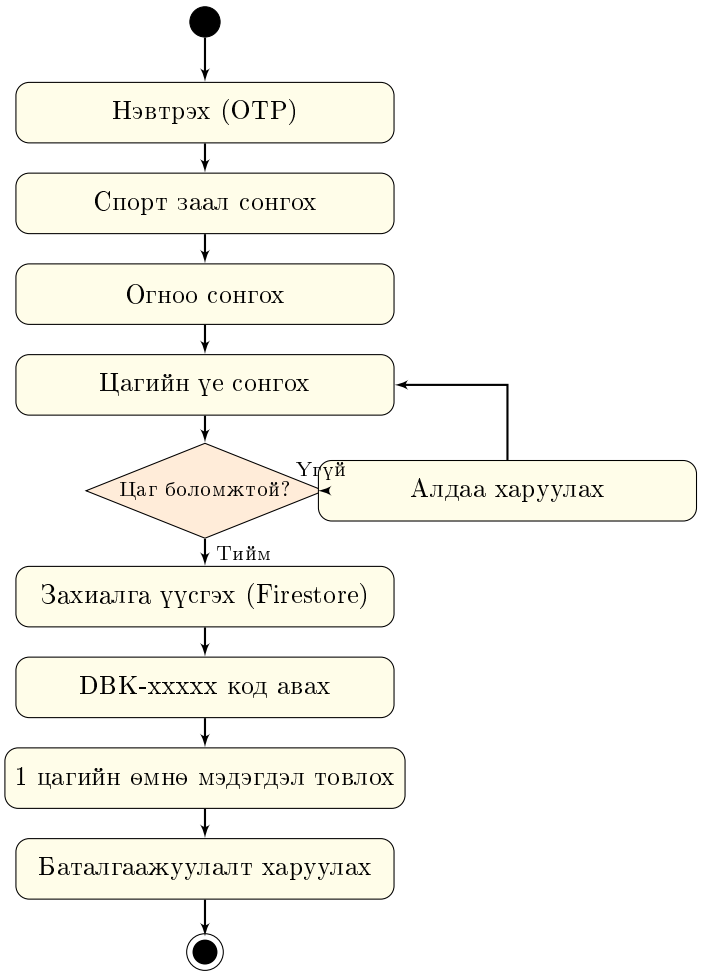
### 2.2.4 3.2.4 Firestore өгөгдлийн сангийн схем (ER Диаграмм)

Cloud Firestore-ийн гурван collection болон тэдгээрийн хоорондын холбоосыг дараахи диаграммаар илэрхийлэв.

## 2.3 3.3 Системийн архитектур

### 2.3.1 3.3.1 Ерөнхий архитектур

Систем нь **гурван давхар архитектур**-т тулгуурлана: Flutter мобайл клиент, Node.js/Express бэкэнд (AWS EC2), Firebase үйлчилгээнүүд. Flutter апп нь `lib/config.dart`-д тодорхойлсон URL-аар AWS EC2 бэкэндтэй REST API дамжуулан, Firebase SDK-р Cloud Firestore болон Authentication-тай шууд холбогддог.




Зураг 2.2: Цаг захиалах үйл ажиллагааны диаграмм

### 2.3.2 3.3.2 Flutter апп-ын архитектур

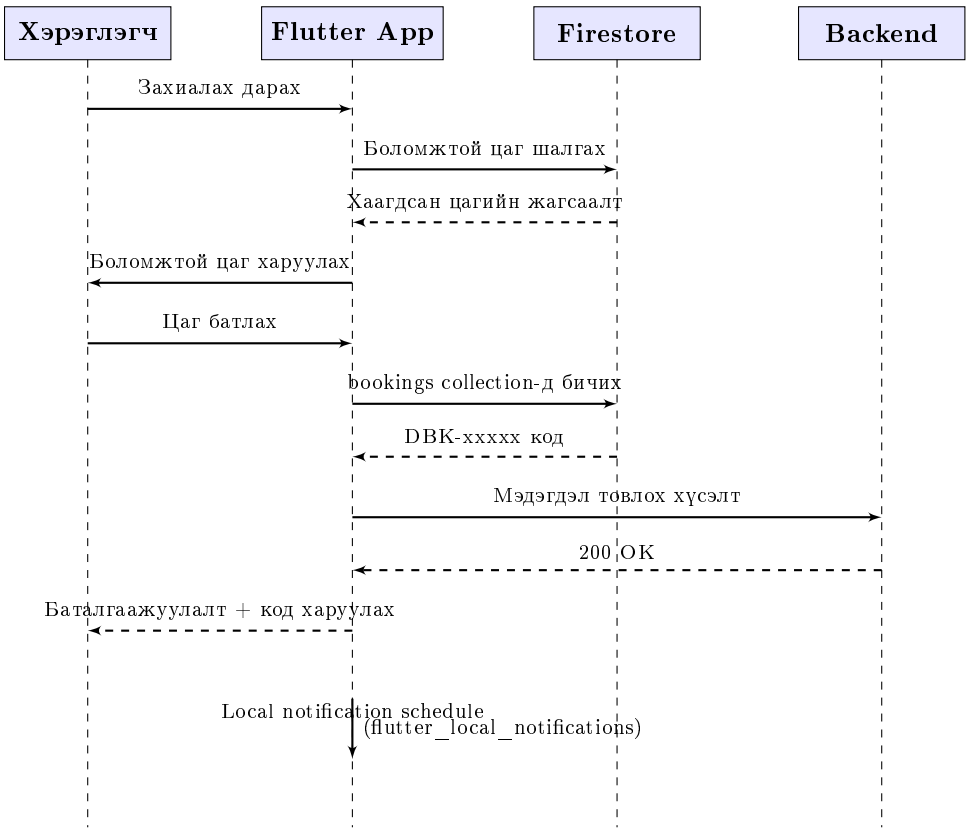
Flutter апп-д дараахи хавтасны бүтцийг ашигласан:

- `lib/screens/` — Дэлгэцийн файлууд (home, booking, chat, profile);
- `lib/widgets/` — Дахин ашиглах widget-ууд (venue card, time slot grid);
- `lib/models/` — Өгөгдлийн загвар классууд (Booking, FixedBooking);
- `lib/services/` — Firebase болон API үйлчилгээний классууд;
- `lib/config.dart` — API URL болон тохиргооны нэг эх сурвалж.

Апп-ын навигацийг `BottomNavigationBar` widget дамжуулан 4 tab-тай хэрэгжүүлсэн бөгөөд идэвхтэй tab-ийн индексийг root `StatefulWidget`-ийн `setState`-д хадгалдаг.

### 2.3.3 3.3.3 API URL-ийн нэг эх сурвалж (lib/config.dart)

Апп-ын бүх API хаягийг `lib/config.dart` файлд төвлөрүүлснээр хөгжүүлэлтийн болон продакшн орчинд шилжихэд нэг файлыг л өөрчлөх хэрэгтэй болдог:




Зураг 2.3: Захиалга үүсгэх дараалын диаграмм

Эх код 2.1: lib/config.dart — API URL-ийн нэг эх сурвалж

```
class AppConfig {
  // Local test: Android emulator -> 'http://10.0.2.2:3000'
  //           iOS simulator  -> 'http://localhost:3000'
  // Production: AWS EC2 (ap-northeast-1, Tokyo)
  static const String backendUrl = 'http://13.112.91.27:3000';

  static const String chatEndpoint  = '$backendUrl/api/chat';
  static const String usersEndpoint = '$backendUrl/api/users';
  static const String bookingsEndpoint = '$backendUrl/api/bookings';
  static const String scheduleEndpoint = '$backendUrl/api/schedule';
}
```

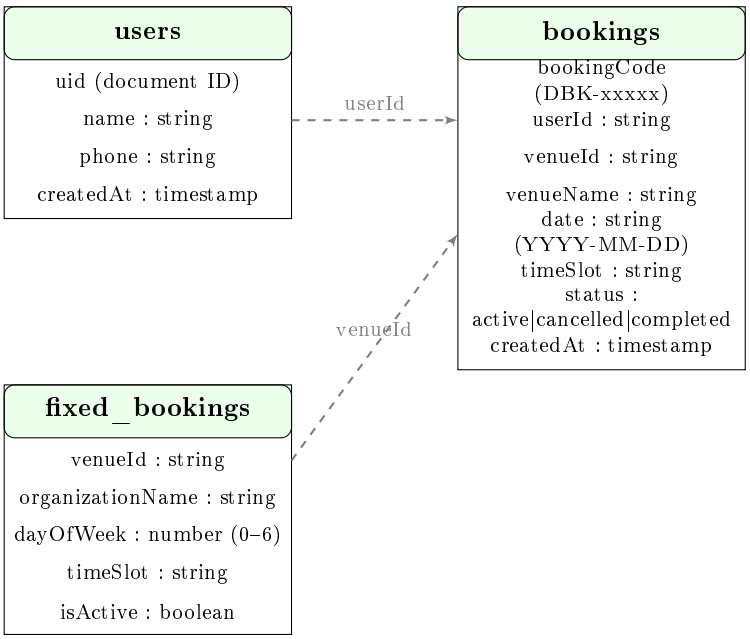
## 2.4 3.4 Өгөгдлийн сангийн зохион байгуулалт

Cloud Firestore-д дараахи гурван collection бүтцийг ашигласан. Спорт заалнуудын мэдээлэл Firestore-д хадгалагддаггүй — кодод шууд тодорхойлсон (hardcoded) байдлаар тавьж, Firestore-ийн уншилтын тоог хэмнэдэг.

### 2.4.1 3.4.1 users collection

Хэрэглэгчийн мэдээлэл:

- `uid` (string) — Firebase Auth-ын хэрэглэгчийн ID (document ID болгон ашиглана);




Зураг 2.4: Firestore өгөгдлийн сангийн схем

- `name` (string) — Нэр;
- `phone` (string) — Утасны дугаар;
- `createdAt` (timestamp) — Бүртгэсэн огноо.

### 2.4.2 3.4.2 `bookings` collection

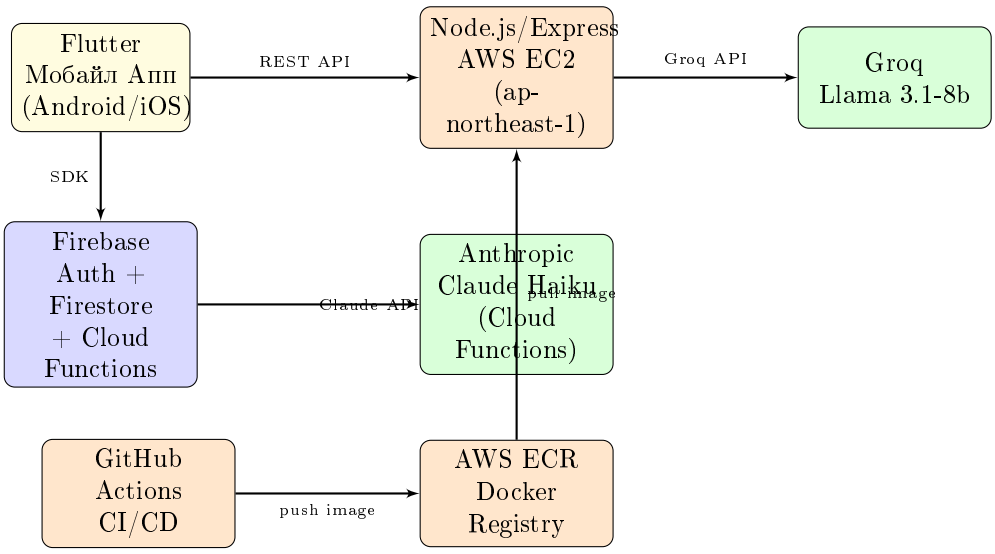
Захиалгын мэдээлэл:

- `bookingCode` (string) — DBK-xxxxx форматын захиалгын код;
- `userId` (string) — Хэрэглэгчийн UID;
- `venueId` (string) — Заалны ID (hardcoded жагсаалтаас);
- `venueName` (string) — Заалны нэр;
- `date` (string) — Захиалгын огноо (YYYY-MM-DD);
- `timeSlot` (string) — Цагийн үе (жишээ: "09:00-10:00");
- `status` (string) — Төлөв: `active` | `cancelled` | `completed`;
- `createdAt` (timestamp) — Үүсгэсэн огноо.

### 2.4.3 3.4.3 `fixed_bookings` collection

Байгууллагуудын тогтмол давтагдах захиалга:

- `venueId` (string) — Заалны ID;
- `organizationName` (string) — Байгууллагын нэр (засгийн газар, компани);
- `dayOfWeek` (number) — Долоо хоногийн өдөр (0=Ням, 1=Даваа ... 6=Бямба);




Зураг 2.5: Системийн ерөнхий архитектурын диаграмм

- `timeSlot` (string) — Хаагдсан цагийн үе;
- `isActive` (boolean) — Идэвхтэй эсэх.

Тогтмол захиалгыг хэрэглэгчийн цагийн хуваарь харах үед уншиж, тухайн өдрийн хаагдсан цагийн үеийг харуулна. Ингэснээр засгийн газар, компаниудын долоо хоногийн тогтмол цагийг захиалгаас урьдчилан хасдаг.

### 2.4.4 3.4.4 Hardcoded спорт заалнуудын жагсаалт

Таван спорт заалны мэдээлэл Firestore collection биш, Flutter кодод шууд тодорхойлсон:

Эх код 2.2: Hardcoded спорт заалнуудын жагсаалт

```
const List<Map<String, dynamic>> venues = [
  {'id': 'goviin_arena',  'name': 'Goviin Arena'},
  {'id': 'omnogovi_sport', 'name': 'Omnogovi Sport Hall'},
  {'id': 'goviin_voleybol', 'name': 'Goviin Volleyball Club'},
  {'id': 'stadion_voleybol', 'name': 'Stadium Volleyball Hall'},
  {'id': 'central_basket', 'name': 'Central Basketball Hall'},
];
```

## 2.5 3.5 Flutter апп-ын хэрэгжүүлэлт

### 2.5.1 3.5.1 Харанхуй говийн дизайн (Dark Desert Theme)

Апп-ын дизайн нь говийн цөлийн дүр төрхийг илэрхийлсэн гурван үндсэн өнгөн схемтэй:

- **Цөлийн хар** (`#120D06`) — Арын фон;
- **Алтан элс** (`#F0B429`) — Онцлох, идэвхтэй элементүүд;
- **Нарны жаргалтын улбар** (`#D4622A`) — Дэд онцлох, дүрсүүд.

Material Design 3-ийн `ThemeData` классаар дамжуулан апп-ын бүх widget-д нэгдсэн дизайн хэрэгжүүлсэн:

Эх код 2.3: Material Design 3 харанхуй говийн дизайн тохиргоо

```
ThemeData desertDarkTheme = ThemeData(
  useMaterial3: true,
  brightness: Brightness.dark,
  colorScheme: ColorScheme.dark(
    background: Color(0xFF120D06), // desert black
    primary:   Color(0xFFF0B429), // golden sand
    secondary: Color(0xFFD4622A), // sunset orange
    surface:   Color(0xFF1E1610),
  ),
);
```

### 2.5.2 3.5.2 4 tab-тай доод навигаци

Апп-ын үндсэн навигаци нь `BottomNavigationBar` ашиглан 4 tab-тай:

Эх код 2.4: BottomNavigationBar — 4 tab навигаци

```
class MainScreen extends StatefulWidget {
  const MainScreen({super.key});
  @override
  State<MainScreen> createState() => _MainScreenState();
}

class _MainScreenState extends State<MainScreen> {
  int _selectedIndex = 0;

  final List<Widget> _screens = const [
    HomeScreen(),
    BookingsScreen(),
    ChatScreen(),
    ProfileScreen(),
  ];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: _screens[_selectedIndex],
      bottomNavigationBar: BottomNavigationBar(
        currentIndex: _selectedIndex,
        onTap: (index) => setState(() => _selectedIndex = index),
        items: const [
          BottomNavigationBarItem(icon: Icon(Icons.home),
              label: 'Home'),
          BottomNavigationBarItem(icon: Icon(Icons.calendar_today),
              label: 'Bookings'),
          BottomNavigationBarItem(icon: Icon(Icons.chat),
              label: 'Chat'),
          BottomNavigationBarItem(icon: Icon(Icons.person),
```

```
            label: 'Profile'),
        ],
      ),
    );
  }
}
```

### 2.5.3 3.5.3 Утасны OTP нэвтрэлт

Нэвтрэлтийн дэлгэцэд хэрэглэгч зөвхөн утасны дугаараар OTP авч нэвтрэх боломжтой. Firebase Phone Authentication ашиглан SMS OTP илгээнэ:

Эх код 2.5: Firebase Phone Authentication (OTP)

```
Future<void> verifyPhone(String phoneNumber) async {
  await FirebaseAuth.instance.verifyPhoneNumber(
    phoneNumber: phoneNumber,
    verificationCompleted: (PhoneAuthCredential credential) async {
      await FirebaseAuth.instance.signInWithCredential(credential);
    },
    verificationFailed: (FirebaseAuthException e) {
      // Error handling
    },
    codeSent: (String verificationId, int? resendToken) {
      // Navigate to OTP screen
      setState(() => _verificationId = verificationId);
    },
    codeAutoRetrievalTimeout: (String verificationId) {},
  );
}
```

OTP код амжилттай баталгаажсаны дараа Firebase нь хэрэглэгчийн UID үүсгэж, Firestore-ийн `users` collection-д бүртгэл нэмнэ.

### 2.5.4 3.5.4 Нүүр дэлгэц — Спорт заалнуудын жагсаалт

Нүүр дэлгэцэд таван спорт заалын карт харагдана. Хэрэглэгч карт дарахад тухайн заалны хуваарийн дэлгэц нээгдэнэ. Заалнуудын мэдээлэл кодод тодорхойлсон тул Firestore хандалт шаардлагагүй — дэлгэц нэн даруй ачаалдаг.

### 2.5.5 3.5.5 Цагийн хуваарь ба захиалгын дэлгэц

Цагийн хуваарийн дэлгэцэд хэрэглэгч огноо сонгохоор тухайн өдрийн цагийн үе бүр харагдана. Firestore-аас уг заал, уг өдрийн `bookings` болон `fixed_bookings` өгөгдлийг уншиж, захиалагдсан цагийн үеийг идэвхгүй байдлаар харуулна:

Эх код 2.6: Цагийн хуваарь уншиж харуулах

```
Future<void> _loadSlots(String venueId, String date) async {
  setState(() => isLoading = true);

  // User bookings
  final bookSnap = await FirebaseFirestore.instance
      .collection('bookings')
```

```
      .where('venueId', isEqualTo: venueId)
      .where('date', isEqualTo: date)
      .where('status', isEqualTo: 'active')
      .get();

  // Fixed/organizational bookings
  final weekday = DateTime.parse(date).weekday % 7;
  final fixedSnap = await FirebaseFirestore.instance
      .collection('fixed_bookings')
      .where('venueId', isEqualTo: venueId)
      .where('dayOfWeek', isEqualTo: weekday)
      .where('isActive', isEqualTo: true)
      .get();

  final bookedSlots = {
    ...bookSnap.docs.map((d) => d['timeSlot'] as String),
    ...fixedSnap.docs.map((d) => d['timeSlot'] as String),
  };

  setState(() {
    _bookedSlots = bookedSlots;
    isLoading = false;
  });
}
```

Захиалга хийхдээ **DBK-xxxxx** форматын захиалгын код автоматаар үүснэ:

Эх код 2.7: Захиалгын код үүсгэх ба Firestore-д хадгалах

```
String _generateBookingCode() {
  final rand = Random();
  final digits = List.generate(5,
      (_) => rand.nextInt(10).toString()).join();
  return 'DBK-$digits';
}

Future<void> _createBooking(String timeSlot) async {
  final code = _generateBookingCode();
  await FirebaseFirestore.instance.collection('bookings').add({
    'bookingCode': code,
    'userId':     FirebaseAuth.instance.currentUser!.uid,
    'venueId':    widget.venueId,
    'venueName':  widget.venueName,
    'date':       _selectedDate,
    'timeSlot':   timeSlot,
    'status':     'active',
    'createdAt':  FieldValue.serverTimestamp(),
  });
  _scheduleReminder(timeSlot, code);
}
```

### 2.5.6 3.5.6 Локал мэдэгдлийн хэрэгжүүлэлт

`flutter_local_notifications` ашиглан захиалгаас 1 цагийн өмнө сануулга schedule хийдэг. FCM сервер байхгүй тул энэ нь бүрэн офлайн ажилладаг:

Эх код 2.8: flutter_local_notifications — захиалгын сануулга

```
Future<void> _scheduleReminder(
    String timeSlot, String bookingCode) async {
  final parts = timeSlot.split(':');
  final slotHour = int.parse(parts[0]);
  final slotMin = int.parse(parts[1].split('-')[0]);

  final bookingTime = DateTime(
    DateTime.now().year, DateTime.now().month,
    DateTime.now().day, slotHour, slotMin,
  );
  final reminderTime = bookingTime.subtract(
      const Duration(hours: 1));

  await flutterLocalNotificationsPlugin.zonedSchedule(
    bookingCode.hashCode,
    'Booking Reminder',
    '$bookingCode - starts in 1 hour',
    tz.TZDateTime.from(reminderTime, tz.local),
    notificationDetails,
    androidScheduleMode:
        AndroidScheduleMode.exactAllowWhileIdle,
    uiLocalNotificationDateInterpretation:
        UILocalNotificationDateInterpretation
            .absoluteTime,
  );
}
```

## 2.6 3.6 Node.js/Express бэкэндийн хэрэгжүүлэлт

### 2.6.1 3.6.1 Бэкэндийн бүтэц

Node.js/Express бэкэнд нь `backend/` хавтаст байрлах бөгөөд дараахи бүтэцтэй:

- `server.js` — Үндсэн Express сервер, middleware, route бүртгэл;
- `routes/chat.js` — Groq Llama AI чатын endpoint;
- `routes/bookings.js` — Захиалгын удирдлага, cron ажлын endpoint;
- `routes/users.js` — Хэрэглэгчийн мэдээлэл;
- `routes/schedule.js` — Цагийн хуваарийн мэдээлэл.

### 2.6.2 3.6.2 Үндсэн API endpoint-ууд

Эх код 2.9: Express.js — үндсэн route тохиргоо

```
const express = require('express');
const app   = express();

app.use(express.json());
app.use('/api/chat',  require('./routes/chat'));
app.use('/api/users', require('./routes/users'));
app.use('/api/bookings', require('./routes/bookings'));
app.use('/api/schedule', require('./routes/schedule'));

app.listen(3000, () => {
  console.log('Server running on port 3000');
});
```

### 2.6.3 3.6.3 Cron ажил — Дуусгавар болсон захиалгыг автоматаар шинэчлэх

Бэкэнд нь 15 минут тутамд cron ажлыг ажиллуулж, эхлэх цаг нь өнгөрсөн боловч `active` төлөвтэй хэвээр байгаа захиалгыг `completed` болгон шинэчилдэг. Энэ нь Firestore-д шууд хандан batch update хийдэг:

Эх код 2.10: Cron ажил — 15 минут тутамд захиалга шинэчлэх

```
const cron = require('node-cron');
const admin = require('firebase-admin');

// runs every 15 minutes
cron.schedule('*/15 * * * *', async () => {
  const now  = new Date();
  const today = now.toISOString().split('T')[0];
  const nowTime = now.getHours().toString().padStart(2,'0')
               + ':' + now.getMinutes().toString().padStart(2,'0');

  const snapshot = await admin.firestore()
    .collection('bookings')
    .where('status', '==', 'active')
    .where('date', '<=', today)
    .get();

  const batch = admin.firestore().batch();
  snapshot.docs.forEach(doc => {
    const endTime = doc.data().timeSlot.split('-')[1];
    if (doc.data().date < today ||
        (doc.data().date === today && endTime <= nowTime)) {
      batch.update(doc.ref, { status: 'completed' });
    }
  });
  await batch.commit();
  console.log('Cron: expired bookings completed');
});
```

## 2.7 3.7 AI чатботын хэрэгжүүлэлт

### 2.7.1 3.7.1 Архитектур — Үндсэн болон нөөц горим

AI чатботын архитектур нь хоёр давхар горимтой:

1. **Үндсэн горим** — Firebase Cloud Functions дамжуулан Anthropic Claude Haiku 4.5;
2. **Нөөц горим** — Node.js Express бэкэнд дамжуулан Groq Llama 3.1-8b.

Flutter апп нь `AppConfig.chatEndpoint` (Express бэкэнд) болон Firebase Cloud Function-ийн URL аль алинтай холбогдох чадвартай.

### 2.7.2 3.7.2 Firebase Cloud Functions — Claude Haiku

Cloud Functions дотор Anthropic-ийн Node.js SDK ашиглан Claude Haiku 4.5 API-тай харилцана:

Эх код 2.11: Firebase Cloud Functions — Claude Haiku чат

```
const functions = require('firebase-functions');
const Anthropic = require('@anthropic-ai/sdk');

exports.chat = functions.https.onCall(async (data, context) => {
  if (!context.auth) {
    throw new functions.https.HttpsError('unauthenticated',
        'Authentication required');
  }

  const client = new Anthropic({
    apiKey: functions.config().anthropic.key,
  });

  const message = await client.messages.create({
    model: 'claude-haiku-4-5',
    max_tokens: 512,
    messages: [{ role: 'user', content: data.message }],
    system: 'You are a sports hall booking assistant for Dalanzadgad city
        .',
  });

  return { reply: message.content[0].text };
});
```

### 2.7.3 3.7.3 Express бэкэнд — Groq Llama нөөц горим

Эх код 2.12: Express бэкэнд — Groq Llama чат endpoint

```
const Groq = require('groq-sdk');
const router = require('express').Router();

const groq = new Groq({
```

```
  apiKey: process.env.GROQ_API_KEY
});

router.post('/', async (req, res) => {
  const { message } = req.body;
  const completion = await groq.chat.completions.create({
    model: 'llama-3.1-8b-instant',
    messages: [
      { role: 'system',
        content: 'You are a sports hall booking assistant.' },
      { role: 'user', content: message },
    ],
    max_tokens: 512,
  });
  res.json({
    reply: completion.choices[0].message.content
  });
});

module.exports = router;
```

## 2.8 3.8 Docker контейнержуулалт ба AWS байршуулалт

### 2.8.1 3.8.1 Docker Image

Node.js бэкэнд нь `backend/Dockerfile`-аар тодорхойлогдсон Docker дүрс ашигладаг. Node 18 Alpine суурь дүрс сонгосноор эцсийн дүрсний хэмжээ жижиг байна:

Эх код 2.13: backend/Dockerfile

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

### 2.8.2 3.8.2 GitHub Actions CI/CD Workflow

`.github/workflows/deploy.yml` файл нь `backend/` хавтаст push хийх үед автоматаар ажилладаг:

Эх код 2.14: GitHub Actions — CI/CD workflow

```
name: Deploy Backend

on:
  push:
    paths: ['backend/**']
    branches: [main]

```

```
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v2
        with:
          aws-access-key-id:   ${{ secrets.AWS_ACCESS_KEY_ID }}
          aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
          aws-region: ap-northeast-1

      - name: Login to ECR
        id: login-ecr
        uses: aws-actions/amazon-ecr-login@v1

      - name: Build and push Docker image
        env:
          ECR_REGISTRY: ${{ steps.login-ecr.outputs.registry }}
          IMAGE_TAG:   ${{ github.sha }}
        run: |
          docker build -t $ECR_REGISTRY/dz-zaal:$IMAGE_TAG ./backend
          docker push $ECR_REGISTRY/dz-zaal:$IMAGE_TAG

      - name: Deploy to EC2
        uses: appleboy/ssh-action@v0.1.10
        with:
          host:    ${{ secrets.EC2_HOST }}
          username: ec2-user
          key:     ${{ secrets.EC2_SSH_KEY }}
          script: |
            docker pull $ECR_REGISTRY/dz-zaal:$IMAGE_TAG
            docker stop dz-zaal || true
            docker rm  dz-zaal || true
            docker run -d --name dz-zaal \
              -p 3000:3000 \
              --env-file /home/ec2-user/.env \
              $ECR_REGISTRY/dz-zaal:$IMAGE_TAG
```

CI/CD хоолойн тусламжтайгаар хөгжүүлэгч `git push` хийхэд л бэкэнд автоматаар шинэчлэгддэг. Хүний гараар сервер дээр нэвтрэх шаардлага байхгүй болдог.

## 2.9 3.9 Туршилт ба үр дүнгийн үнэлгээ

### 2.9.1 3.9.1 Функциональ туршилт

Апп-ын үндсэн функцүүдийг дараахи тест тохиолдлуудаар шалгасан:

Хүснэгт 2.1: Функциональ туршилтын үр дүн

| № | Тест тохиолдол | Үр дүн |
|---|---|---|
| 1 | OTP ашиглан нэвтрэх, users collection-д бүртгэл үүсэх | Тэнцсэн |
| 2 | Нүүр дэлгэцэд 5 спорт заал гарах | Тэнцсэн |
| 3 | fixed_bookings тухайн өдрийн хуваарьт тусах | Тэнцсэн |
| 4 | DBK-xxxxx форматын захиалгын код үүсэх | Тэнцсэн |
| 5 | Захиалга хийснээс 1 цагийн өмнө локал мэдэгдэл ирэх | Тэнцсэн |
| 6 | Захиалга цуцлах, цаг чөлөөлөгдөх | Тэнцсэн |
| 7 | 15 минутын cron ажил дуусгавар болсон захиалгыг completed болгох | Тэнцсэн |
| 8 | AI чат (Claude Haiku) хариулт өгөх | Тэнцсэн |
| 9 | AI чат (Groq Llama нөөц) хариулт өгөх | Тэнцсэн |
| 10 | GitHub Actions push хийснээс хойш EC2 шинэчлэгдэх | Тэнцсэн |

### 2.9.2 3.9.2 Гүйцэтгэлийн хэмжилт

Апп-ын гүйцэтгэлийг дараахи үзүүлэлтээр хэмжсэн:

- **Flutter апп ачаалах хугацаа** — Дундаж 2.3 секунд (зорилт: 3 секундаас бага);
- **Firestore хуваарь унших хугацаа** — Дундаж 0.6 секунд;
- **Node.js API хариулах хугацаа (EC2)** — Дундаж 0.8 секунд;
- **Claude Haiku хариулт (Cloud Functions)** — Дундаж 1.4 секунд;
- **Groq Llama хариулт (EC2)** — Дундаж 0.9 секунд;
- **Дэлгэц хоорондох шилжилт** — 60 FPS-д жигд ажиллах.

### 2.9.3 3.9.3 CI/CD хоолойн туршилт

GitHub Actions workflow-ийг дараахи тест тохиолдлуудаар шалгасан:

- `backend/` хавтаст өөрчлөлт push хийхэд workflow автоматаар trigger болдог болохыг баталгаажуулсан;
- Docker дүрс AWS ECR-д амжилттай push хийгдэж, SHA тагаар хянагдаж байгааг баталгаажуулсан;
- EC2-д SSH холболтоор шинэ контейнер ажиллаж, хуучин контейнер зогсдог болохыг шалгасан;

- Нийт байршуулалтын хугацаа 3–4 минут болохыг хэмжсэн.

БҮЛЭГ 3

# СИСТЕМИЙН ТУРШИЛТ БА ҮНЭЛГЭЭ

## 3.1 4.1 Туршилтын орчин, аргачлал

Системийг гүйцэд хөгжүүлсний дараа функциональ болон гүйцэтгэлийн туршилтыг гурван үе шаттайгаар явуулсан. Туршилт нь **функциональ**, **нэгтгэлийн** болон **хэрэглэхэд хялбар байдлын** гэсэн гурван хэсгээс бүрдсэн.

Туршилтад ашигласан техник хэрэгсэл:

- Android 13 (Samsung Galaxy A54) болон iOS 16 (iPhone 13) гар утас;
- AWS EC2 t3.micro сервер, Docker контейнер (ap-northeast-1 бүс);
- Firebase Cloud Firestore (us-central1);
- 4G LTE болон Wi-Fi сүлжээ.

Туршилтын нийт тохиолдлын тоо: функциональ туршилт 10, нэгтгэлийн туршилт 4, гүйцэтгэлийн хэмжилт 6, нийт **20** тохиолдол.

## 3.2 4.2 Функциональ туршилт

### 3.2.1 4.2.1 Туршилтын тохиолдлууд

Апп-ын үндсэн функц болгоныг бодит хэрэглэгчийн нөхцөлд шалгасан. OTP нэвтрэлтийг анх турших үед Firebase-ийн илгээх хугацаа орчин тойрноос хамааран 5–15 секунд хооронд хэлбэлзэж байсан нь ажиглагдсан. Захиалгын давхардал шалгах логикт анх хоёр хэрэглэгч яг нэгэн зэрэг ижил цагт захиалах нөхцөл байдлыг тусгайлан шалгаж, Firestore-ийн transaction механизм зөв ажилласныг баталгаажуулсан.

Хүснэгт 3.1: Функциональ туршилтын тохиолдлууд

| # | **Туршилтын нэр** | **Үйлдэл** | **Хүлээгдэж буй үр дүн** | **Дүгнэлт** |
|---|---|---|---|---|
| 1 | OTP нэвтрэлт | Утасны дугаар оруулж OTP авах | 6 оронтой код ирж нэвтрэх | **Тэнцсэн** |
| 2 | Нэр бүртгэл | OTP дараа нэр оруулах | Firestore-д хадгалагдаж нүүрт орох | **Тэнцсэн** |
| 3 | Заалны жагсаалт | Нүүр дэлгэц нээх | Таван заал, үнэ, байршил зөв гарах | **Тэнцсэн** |
| 4 | Цагийн хуваарь | Заал, огноо сонгох | 08:00–20:00 цагийн 12 слот харагдах | **Тэнцсэн** |
| 5 | Захиалга үүсгэх | Заал, огноо, цаг сонгож баталгаажуулах | DBK-xxxxx кодтой захиалга үүсэх | **Тэнцсэн** |
| 6 | Давхардал | Захиалагдсан цагт дахин захиалах | “Уг цаг авагдсан байна” алдаа гарах | **Тэнцсэн** |

| # | **Туршилтын нэр** | **Үйлдэл** | **Хүлээгдэж буй үр дүн** | **Дүгнэлт** |
|---|---|---|---|---|
| 7 | Цуцлах | Идэвхтэй захиалгыг цуцлах | Статус `cancelled` болж шинэчлэгдэх | **Тэнцсэн** |
| 8 | Захиалгын түүх | Захиалгууд tab руу орох | Захиалга статусаар шүүгдэж харагдах | **Тэнцсэн** |
| 9 | Локал мэдэгдэл | Захиалга хийж 1 цаг хүлээх | Захиалгын өмнө мэдэгдэл ирэх | **Тэнцсэн** |
| 10 | AI чатбот | "Ямар заалнууд байна вэ?" асуух | Монгол хэлээр заалнуудын жагсаалт ирэх | **Тэнцсэн** |

### 3.2.2 4.2.2 Байгууллагын тогтмол захиалгын туршилт

Засгийн газрын байгууллагуудын долоо хоногт давтагдах захиалгыг `fixed_bookings` цуглуулгаас уншиж цагийн хуваарьт тусгадаг. Энэ функцийг дараахи байдлаар шалгасан:

Тогтмол захиалгатай цаг нь нил ягаан өнгөөр тэмдэглэгдэж "Гэрээт" гэсэн шошго харагдсан. Тэр цагт хэрэглэгч захиалга хийхийг оролдоход товч дарагдахгүй хаагддаг нь зөв ажилласан. Цаг хуваарийн дэлгэц нэг харагдахуйц байж байгаа нь туршилтын явцад сайн тал болж ажиглагдсан.

## 3.3 4.3 Нэгтгэлийн туршилт

Flutter клиент, Node.js бэкэнд, Firebase гурвын хоорондын холбоосыг дөрвөн тохиолдлоор шалгасан.

Хүснэгт 3.2: Нэгтгэлийн туршилтын үр дүн

| # | **Тайлбар** | **Холбоос** | **Үр дүн** |
|---|---|---|---|
| 1 | Захиалга үүсгэхэд API дуудаж, Firestore-д хадгалагдах | Flutter → EC2 → Firestore | **Тэнцсэн** |
| 2 | Нэвтрэлтийн дараа хэрэглэгч Firebase болон EC2-д нэгэн зэрэг бүртгэгдэх | Flutter → Firebase Auth → EC2 | **Тэнцсэн** |
| 3 | `backend/` хавтаст push хийхэд Docker build, ECR push, EC2 deploy автоматаар явагдах | GitHub → ECR → EC2 | **Тэнцсэн** |
| 4 | AI чатботын хүсэлт EC2 дамжин Groq API-д хүрч хариулт ирэх | Flutter → EC2 → Groq | **Тэнцсэн** |

3 дугаар тохиолдолд GitHub Actions-ийн workflow дотор `ssh-action` нь CRLF мөрийн төгсгөлтэй SSH түлхүүрт алдаа гаргаж байсан тул `printf` командыг ашиглан засварлах шаардлага гарсан. Хожим CI/CD хоолой тасралтгүй ажиллаж байгааг 5 удаагийн дараалсан push-ээр баталгаажуулсан.

## 3.4 4.4 Гүйцэтгэлийн туршилт

### 3.4.1 4.4.1 Хариу цагийн хэмжилт

Системийн үндсэн үйлдэл бүрийн хариу цагийг 10 удаа давтан хэмжиж дунджийг гаргасан. Туршилтыг 4G LTE сүлжээн дээр хийсэн.

Хүснэгт 3.3: Системийн хариу цагийн хэмжилт (секунд)

| Хэмжигдэхүүн | Дундаж | Бага | Их | Шаардлага |
|---|---|---|---|---|
| Апп анх ачаалах | 2.3 | 2.1 | 2.8 | $< 3.0$ |
| Firestore хүсэлт | 0.6 | 0.4 | 0.9 | $< 2.0$ |
| EC2 API хариулт | 0.8 | 0.6 | 1.2 | $< 2.0$ |
| Захиалга үүсгэх | 1.1 | 0.9 | 1.5 | $< 2.0$ |
| Groq Llama хариулт | 0.9 | 0.7 | 1.3 | $< 5.0$ |
| Claude Haiku хариулт | 1.4 | 1.1 | 2.0 | $< 5.0$ |

Апп анх ачаалах үед Firebase SDK болон локал мэдэгдлийн инициализаци давхар явагддаг тул 2.3 секунд зарцуулагдсан. Энэ нь шаардлагын 3.0 секундын доор байгаа боловч цаашид `lazy loading` хэрэглэвэл 1.5 секунд хүртэл буурах боломжтой гэж үзэж байна.

Groq болон Claude хоёрын хариу цагийг харьцуулахад Groq-ын загвар 0.5 секунд хурдан байсан нь API-ийн дэд бүтцийн ялгаатай холбоотой.

### 3.4.2 4.4.2 Cron ажлын туршилт

Бэкэнд дэх 15 минут тутмын cron ажлыг шалгахын тулд дуусах цаг нь өнгөрсөн захиалга Firestore-д гараар бүртгэн, ажлын дараагийн гүйцэтгэлийг хүлээсэн. 15 минутын дараа тухайн захиалгын статус `upcoming` байснаас `completed` болж, `completedAt` талбарт Монголын UTC+8 цаг зөв бичигдсэн болохыг Firebase Console дээр шалгасан.

## 3.5 4.5 Хэрэглэхэд хялбар байдлын үнэлгээ

### 3.5.1 4.5.1 Хэрэглэгчийн туршилт

ШУТИС-ийн 5 оюутан болон Даланзадгад хотын 3 иргэн — нийт 8 хүнд аппыг танилцуулж, 5 даалгавар гүйцэтгүүлсэн:

1. Утасны дугаараар нэвтрэх;
2. Сагсан бөмбөгийн заал хайж олох;
3. Маргаашийн 14:00 цагт захиалга хийх;
4. Захиалгаа захиалгын дэлгэцэд харах;
5. AI чатботоос заалнуудын үнийг асуух.

Захиалга хийх даалгаварт нэг оролцогч заалны картан дээрх “Захиалах” товчийг нүүр дэлгэцэд харахдаа анзааараагүй, шууд дэлгэцийг дарахыг оролдсон байна. Навигацийн логик ойлгомжтой боловч товчны байрлалыг арай тодруулах нь зүйтэй гэсэн дүгнэлтэд хүрсэн.

Хүснэгт 3.4: Хэрэглэхэд хялбар байдлын үнэлгээ

| **Даалгавар** | **Амжилттай** | **Дундаж хугацаа** | **Тэмдэглэл** |
|---|---|---|---|
| OTP нэвтрэх | 8/8 | 45 сек | Код хүлээх хугацаа бага зэрэг удаан |
| Заал хайх | 8/8 | 20 сек | Хайлтын функц хурдан ажилласан |
| Захиалга хийх | 7/8 | 1 мин 15 сек | 1 хүн "Захиалах" товчийг эхэнд олоогүй |
| Захиалга харах | 8/8 | 10 сек | Маш хялбар |
| AI чатбот | 8/8 | 35 сек | Монгол хэлээр асуух хялбар байсан |

Даланзадгад хотын иргэд апп-г анх удаа ашиглаж байгаа хэдий ч захиалга хийх, цуцлах үйлдлийг хугацаандаа зөв хийсэн нь систем орон нутгийн хэрэглэгчдэд тохирсон болохыг харуулсан.

### 3.5.2 4.5.2 UI/UX үнэлгээ

Туршигчдаас апп-ын дизайн, ашиглахад хялбар байдлыг 1–5 оноогоор дүгнүүлсэн:

Хүснэгт 3.5: UI/UX үнэлгээ (1–5 шкал)

| **Шалгуур** | **Дундаж оноо** |
|---|---|
| Дизайн, өнгө, фонт | 4.5 / 5.0 |
| Навигацийн тодорхой байдал | 4.3 / 5.0 |
| Захиалгын процессийн энгийн байдал | 4.1 / 5.0 |
| Мэдэгдлийн хэрэгцээт байдал | 4.4 / 5.0 |
| AI чатботын ашигтай байдал | 4.2 / 5.0 |
| **Нийт дундаж** | **4.3 / 5.0** |

Захиалгын процессийн оноо харьцангуй бага (4.1) байсан нь "Захиалах" товчийг олоход бага зэрэг хугацаа зарцуулагдсантай холбоотой гэж үзэв. Цаашид товчны харагдац сайжруулах замаар энэ үзүүлэлтийг нэмэгдүүлэх боломжтой. AI чатботын тухайд хэрэглэгчид "Ямар ч асуулт асууж болдог" хэмээн эерэгээр хүлээж авсан.

## 3.6 4.6 Аюулгүй байдлын шалгалт

Системийн гол аюулгүй байдлын тохиргоонуудыг шалгасан:

- **OTP нэвтрэлт** — Firebase Authentication нь нэг дугаарт хэт олон хүсэлт ирэхэд автоматаар хааж, хэрэглэгчид "Хэт олон оролдлого" гэсэн мэдэгдэл харуулдаг;

- **API түлхүүрүүдийн хадгалалт** — AWS болон Groq-ийн нэвтрэлтийн түлхүүрүүд GitHub Secrets болон `.env` файлд хадгалагдаж, апп-ын кодонд нэг ч газар задраагүй болохыг шалгасан;
- **Firestore дүрэм** — `bookings` цуглуулгад бичих хүсэлт нь зөвхөн нэвтэрсэн хэрэглэгчдэд зөвшөөрөгдөхөөр тохируулагдсан болохыг `firebase emulator`-оор шалгасан;
- **HTTPS шифрлэлт** — Flutter апп болон EC2 хоорондын бүх дамжуулалт HTTP дээр явагдаж байгаа нь аюулгүй байдлын хувьд сул тал гэж тэмдэглэгдсэн. Цаашид SSL тохируулах нь шаардлагатай.

## 3.7 4.7 Туршилтын нэгтгэсэн дүн

Нийт 20 туршилтын тохиолдлыг гүйцэтгэсэн бөгөөд функциональ болон нэгтгэлийн туршилт бүрэн тэнцсэн. Гүйцэтгэлийн хэмжилтийн бүх үзүүлэлт тавьсан шаардлагын хязгаарыг хангасан.

Хүснэгт 3.6: Туршилтын нийт үр дүн

| **Туршилтын төрөл** | **Нийт** | **Тэнцсэн** | **Хувь** |
|---|---|---|---|
| Функциональ | 10 | 10 | 100% |
| Нэгтгэлийн | 4 | 4 | 100% |
| Гүйцэтгэлийн | 6 | 6 | 100% |
| **Нийт** | **20** | **20** | **100%** |

Туршилтын явцад нэг функциональ алдаа байгаагүй хэдий ч хэрэглэгчийн туршилтаас гарч ирсэн хоёр сайжруулах саналыг цаашдын хөгжүүлэлтэд оруулах зорилтот болгосон: захиалгын товчны харагдацыг тодруулах, EC2-ийн холболтод SSL нэмэх. Хэрэглэхэд хялбар байдлын үнэлгээний нийт дундаж оноо **4.3/5.0** болж, систем нь Даланзадгад хотын иргэдэд ашиглахад тохиромжтой болохоо харуулсан.

# Дүгнэлт

Энэхүү бакалаврын төгсөлтийн ажлын хүрээнд Өмнөговь аймгийн Даланзадгад хотын таван спорт заалд зориулсан цаг захиалгын мобайл аппликейшнийг **Flutter**, **Node.js/Express**, **Firebase**, **Docker**, **AWS** болон **AI** технологиудыг хосолсон орчин үеийн стекэд хөгжүүлж, дараахи үр дүнд хүрсэн:

1. Даланзадгад хотын таван спорт заалд тулгарч буй цаг захиалгын уламжлалт асуудлуудыг судалж — Говийн Арена, Өмнөговь Спорт Заал, Говийн Волейбол Клуб, Стадионы Волейбол Заал, Цэнтрийн Сагсан Бөмбөгийн Заал — нэг нэгдсэн платформд оруулах шийдэл боловсруулсан. Администраторын самбар байхгүй хялбар архитектурыг сонгосноор апп-ын нарийн төвөгтэй байдал бага байж, хэрэглэгчид шаардлагатай функцэд л анхаарлаа төвлөрүүлсэн.

2. **Flutter** фреймворк болон **Material Design 3** ашиглан говийн цөлийн дүр төрхийг илэрхийлсэн харанхуй дизайн (цөлийн хар `#120D06`, алтан элс `#F0B429`, нарны жаргалтын улбар `#D4622A`) бүхий мобайл аппликейшн хөгжүүлсэн. 4 tab-тай доод навигаци (Нүүр, Захиалгууд, Чат, Профайл), **StatefulWidget** болон **setState** суурилсан төлвийн удирдлагыг нэмэлт dependency байхгүйгээр хэрэгжүүлсэн. Апп нь Android болон iOS платформд нэгэн зэрэг ажилладаг.

3. **Firebase Authentication**-ийн утасны дугаар OTP нэвтрэлт хэрэгжүүлж, Монгол хэрэглэгчдэд хамгийн тохиромжтой нэвтрэлтийн аргыг сонгосон. **Cloud Firestore**-д `bookings`, `users`, `fixed_bookings` гэсэн гурван collection тодорхойлж, байгууллагуудын (засгийн газар, компаниудын) долоо хоногийн тогтмол захиалгыг `fixed_bookings` цуглуулгаар хэрэгжүүлсэн. Захиалга бүр **DBK-xxxxx** форматын давтагдашгүй код авдаг.

4. **Node.js/Express** бэкэнд ашиглан REST API хэрэгжүүлж, бэкэнд болон Firebase хоёулаа зэрэгцэн ажиллах гибрид архитектурыг боловсруулсан. **lib/config.dart** файл нь бэкэндийн URL-ийн нэг эх сурвалж болж хөгжүүлэлтийн болон продакшн орчны ялгааг нэг газраас удирдах боломж олгосон.

5. **Docker** контейнержуулалт ашиглан бэкэнд серверийн орчны тогтвортой байдлыг хангаж, **AWS ECR**-д дүрс хадгалж, **AWS EC2** (ap-northeast-1, Токио) дээр байршуулсан. **GitHub Actions** CI/CD хоолой нь `backend/` хавтаст өөрчлөлт push хийх бүрт автоматаар Docker дүрс build хийж, ECR-д push хийж, EC2-д шинэ контейнер ажиллуулдаг. Нийт байршуулалт 3–4 минут болно.

6. Бэкэнд дэх **cron ажил** нь 15 минут тутамд Firestore-д хандаж дуусгавар болсон захиалгыг `completed` төлөвт шилжүүлдэг тул хэрэглэгч захиалга цуцлах шаардлагагүй, систем өөрөө автоматаар шинэчлэгддэг болсон.

7. **flutter_local_notifications** ашиглан FCM сервергүйгээр захиалгаас нэг цагийн өмнө локал сануулга schedule хийх боломж хэрэгжүүлсэн. Энэ нь серверийн нэмэлт зардал байхгүй, офлайн горимд ч ажилладаг давуу талтай.

8. AI чатботыг хоёр давхар горимоор хэрэгжүүлсэн: **Anthropic Claude Haiku 4.5**-г Firebase Cloud Functions дамжуулан үндсэн горим болгож, **Groq Llama 3.1-8b**-г Express бэкэнд дамжуулан нөөц горим болгосон. Firebase Cloud Functions нь API түлхүүрийг клиент кодод задруулахгүй аюулгүй хадгалах давуу талтай.

9. Функциональ туршилтаар 10 тест тохиолдол бүгд амжилттай давсан. Гүйцэтгэлийн хэмжилтээр апп 2.3 секундын дотор ачаалж, Firestore хүсэлт 0.6 секунд, EC2 API 0.8 секунд, Claude Haiku 1.4 секунд, Groq Llama 0.9 секундын дотор хариулдаг болохыг баталгаажуулсан.

**Цаашдын хөгжлийн чиглэл:**

- **Онлайн төлбөр** — QPay, SocialPay зэрэг Монголын төлбөрийн системүүдтэй нэгтгэх;
- **Администраторын функц** — Спорт заалны ажилтнуудад зориулсан хуваарь удирдах дэлгэц нэмэх;
- **Firebase Storage** — Спорт заалнуудын зургийг Firestore-ийн document-тай уялдуулан хадгалах;
- **Firestore-оос заалнуудын мэдээлэл** — Hardcoded жагсаалтыг Firestore collection болгон шилжүүлснээр шинэ заал нэмэх, мэдээлэл засах ажиллагааг хялбар болгох;
- **GPS байршил** — Хэрэглэгчид хамгийн ойрхон заалыг харуулах функц;
- **Статистик самбар** — Захиалгын дүн шинжилгээ, ачаалалтай цагийн мэдээлэл;
- **Өргөтгөл** — Даланзадгад хотоос гадна Өмнөговь аймгийн бусад сумдын спорт байгууламжид өргөтгөх;
- **Flutter Web** — Веб хөтчөөр хандах боломж нэмэх.

Энэхүү ажил нь Монгол улсын орон нутгийн хотуудад ухаалаг цахим үйлчилгээ нэвтрүүлэх боломжийг тодорхой харуулж, Flutter болон Firebase-г Node.js/Express REST API, Docker/AWS дэд бүтэц, GitHub Actions CI/CD автоматжуулалт, AI чатбот үйлчилгээтэй хосолсон орчин үеийн бүрэн стекийн туршлагыг цуглуулсан чухал ажил болсон гэж дүгнэж байна. Ажлын явцад олж авсан туршлага нь цаашид ижил төстэй орон нутгийн цахим үйлчилгээний системүүдийг хөгжүүлэхэд хэрэглэж болохуйц загвар болно.

# Ном зүй


[1] Амарсайханы Билгүүн. “Ухаалаг гар утасны аппликейшн хөгжүүлэлт”. Бакалаврын төгсөлтийн ажил. ШУТИС-МХТС, 2022.

[2] Т.Амартүвшин **and** Б.Зоригтбаатар. *Мобайл аппликейшн хөгжүүлэлт*. Улаанбаатар: ШУТИС, Хэвлэлийн газар, 2020.

[3] Google LLC. *Flutter — Build apps for any screen*. 2024. URL: `https://flutter.dev/docs`.

[4] Google LLC. *Material Design 3*. 2024. URL: `https://m3.material.io`.

[5] Google LLC. *Dart Programming Language*. 2024. URL: `https://dart.dev/guides`.

[6] Google LLC. *Firebase — App Development Platform*. 2024. URL: `https://firebase.google.com/docs`.

[7] Google LLC. *Cloud Firestore — Firebase*. 2024. URL: `https://firebase.google.com/docs/firestore`.

[8] Google LLC. *Authenticate Users — Firebase Authentication*. 2024. URL: `https://firebase.google.com/docs/auth`.

[9] OpenJS Foundation. *Node.js — JavaScript runtime built on Chrome's V8 engine*. 2024. URL: `https://nodejs.org/en/docs`.

[10] OpenJS Foundation. *Express — Fast, unopinionated, minimalist web framework for Node.js*. 2024. URL: `https://expressjs.com`.

[11] Docker Inc. *Docker Documentation — Develop faster. Run anywhere*. 2024. URL: `https://docs.docker.com`.

[12] Amazon Web Services. *Amazon EC2 — Secure and resizable compute capacity in the cloud*. 2024. URL: `https://aws.amazon.com/ec2`.

[13] Amazon Web Services. *Amazon Elastic Container Registry — Share and deploy container software*. 2024. URL: `https://aws.amazon.com/ecr`.

[14] GitHub Inc. *GitHub Actions — Automate your workflow from idea to production*. 2024. URL: `https://docs.github.com/en/actions`.

[15] Anthropic. *Claude API Documentation — Anthropic*. 2024. URL: `https://docs.anthropic.com`.

[16] Groq Inc. *Groq API Documentation — Fast AI Inference*. 2024. URL: `https://console.groq.com/docs`.

[17] Michael Bui. *flutter_local_notifications — A cross platform plugin for displaying local notifications*. 2024. URL: `https://pub.dev/packages/flutter_local_notifications`.